嵌入式芯片与系统
设计竞赛

2026 嵌入式 FPGA 大赛

FPGA 创新设计赛道 · AMD 平台 · 自选赛题（初级组）

基于 Z7 LITE 7010 的六级 RISC V CPU 与 CRC32 加速器设计设计报告

团队编号：52052
团队名称：单枪匹马
队员：范文轩（队长）、李元宏
日期：2026 年 11 月 4 日

目录

摘要 3
第一部分 作品概述 3
1.1 功能与特性 3
1.2 应用领域 4
1.3 主要技术特点 4
1.4 主要性能指标 4
1.5 主要创新点 5
1.6 设计流程 5
第二部分 系统组成及功能说明 5
2.1 整体介绍 5
2.1.1 设计背景与目标 5
2.1.2 系统架构与软硬件划分 6
2.2 硬件系统介绍 9
2.2.1 硬件整体介绍 9
2.2.2 机械设计介绍 10
2.2.3 电路各模块介绍 10
2.3 软件系统介绍 30
2.3.1 软件整体介绍 30
2.3.2 软件各模块介绍 30
2.3.3 从零开始复现 31
2.3.4 AI 辅助设计与可复用方法 38
2.3.5 术语与作品配套文件 39
第三部分 完成情况及性能参数 41
3.1 整体介绍 41
3.2 工程成果 42
3.2.1 机械成果 42
3.2.2 电路成果 42
3.2.3 软件成果 43
3.3 特性成果 44
3.3.1 功能与故障验证 44
3.3.2 实现与时序 45
3.3.3 软件基线与硬件完整调用 46
3.3.4 同配置资源对照 47
3.3.5 验证数据与证据对应 49
第四部分 总结 49
4.1 可扩展之处 49
4.2 心得体会 49
第五部分 参考文献 50

基于 Z7 LITE 7010 的六级 RISC V CPU 与 CRC32 加速器设计

摘要

本作品面向嵌入式数据差错检测，在 Z7-LITE 7010 开发板的可编程逻辑中实现六级顺序单发射 RISC-V 软核与 CRC32 加速器。处理器执行 RV32I 整数、分支与访存指令，并通过 Zicsr 指令访问机器模式控制状态寄存器；系统采用 IF、ID、EX0、EX1、MEM、WB 六级流水线，配置 16 KiB 指令 ROM、64 KiB 数据 RAM，以及静态分支预判、三级前递和访存停顿控制。裸机 C 程序通过 MMIO 调用 CRC-32/ISO-HDLC 引擎，计算结果及周期数据经结果邮箱和 JTAG/VIO 传送到主机。

系统在 XC7Z010CLG400-1 上以 50 MHz 运行，布局布线后的建立、保持和脉宽裕量分别为+2.824 ns、+0.044 ns 和+7.000 ns。12 组测试数据经三次 VIO 辅助复位重跑，共 36 项 CRC 结果与主机独立参考一致。4096 字节输入的查表软件计算耗时 69653 周期，硬件完整调用耗时 10295 周期，加速比为 6.765712 倍；对应资源对照的全 SoC 净增量为 430 LUT 和 198 FF。报告说明系统实现、验证方法和测量结果，并给出从环境安装、源码构建到配套板卡运行与数据导出的操作流程。

关键词：RISC-V；六级流水线；Zynq-7010；CRC32；MMIO；JTAG/VIO；软硬件协同

第一部分 作品概述

1.1 功能与特性

本作品在 Z7-LITE_ES1 的 XC7Z010CLG400-1 可编程逻辑中实现六级顺序单发射 RV32I＋Zicsr 软核与 CRC32 加速器。系统以 50 MHz 运行，配置 16 KiB 指令 ROM、64 KiB 数据 RAM、CRC32 MMIO 外设和 128 字结果邮箱。裸机 C 程序完成测试数据生成、软件查表 CRC、硬件调用与周期采样；主机经

JTAG/VIO 读取结果并用独立 zlib 参考核对。板级三次复位、六次稳定扫描的 36 项 CRC 全部匹配，失败计数为 0。

1.2 应用领域

面向嵌入式存储、通信和文件数据的差错检测，以及 FPGA 软核教学、处理器外设集成和软硬件协同测量。作品以 CRC 校验为实例，展示处理器执行、MMIO 调用、板级采集与独立参考比对的完整流程，并量化不同输入长度下的调用收益。当前验证范围为片上 ROM/RAM 与 JTAG 结果读取；应用场景不代表已接入外部通信、存储设备或操作系统。

1.3 主要技术特点

处理器采用 IF、ID、EX0、EX1、MEM、WB 六级流水线，以三级前递、Load-use 停顿和 save 保持处理数据相关；BRAM 输出寄存器配合两拍读取。CRC-32/ISO-HDLC 引擎每拍处理一个字节，32 位缓冲承接 MMIO 写入；副作用只在 valid/ready 完成沿发生。性能评估区分软件、硬件完整调用和引擎会话三种周期，主要加速比包含配置、搬运、轮询与取结果开销。

1.4 主要性能指标

项目	结果
时钟与存储	50 MHz；ROM 16 KiB；RAM 64 KiB
板级正确性	3 次复位；36/36 CRC；失败 0
4096 字节周期	软件 69653；硬件完整调用 10295
完整调用加速比	6.765712 倍
资源	6560 LUT；7892 FF；20 BRAM；0 DSP
时序裕量 ns	WNS 2.824；WHS 0.044；WPWS 7.000

1.5 主要创新点

① 采用完整调用计时，量化配置、供数与读取成本，并保留短输入不加速的结果。② 在同器件、时钟、CPU 和存储容量下比较两个实际实现，给出全 SoC 资源净差。③ 以固定邮箱、三轮复位和独立 zlib 参考建立板级结果核对流程。

1.6 设计流程

明确处理器、外设和存储契约；完成 CRC RTL 与裸机固件；生成独立参考并执行单元、存储、整机和故障测试；生成 IP、综合及布局布线；下载匹配 BIT/LTX；三轮复位采集并核对邮箱；汇总性能、资源、AI 协作记录与复现步骤。

第二部分 系统组成及功能说明

2.1 整体介绍

2.1.1 设计背景与目标

最开始 Vivado 工程选用 Kintex-7 器件 xc7k325tffg676-2，并在租赁开发板上完成 160 MHz 上板验证。原板卡租赁到期后，我们只能将 CPU RTL 迁移至自有 Z7-LITE_ES1 开发板（XC7Z010CLG400-1），并集成 CRC32 加速器。实现采用板载 50 MHz 时钟，系统工作频率设为 50 MHz；本报告中的板级测试、性能和资源结果均对应迁移后的 50 MHz 系统。

RISC-V 指令集具有开放、模块化和可扩展的特点，适合作为 FPGA 软核的教学、科研与系统集成平台。在嵌入式系统中，CRC 被广泛用于存储、通信和文件校验；软件查表实现会占用大量 CPU 周期，而硬件 CRC 引擎的收益又容易被配置、数据搬运和轮询开销抵消。因此，一个有意义的设计不仅要实现 CRC 引擎，还要把 CPU、总线接口、固件、板级调试和端到端测量做完整。

本作品围绕“RISC-V 处理器执行程序、MMIO 外设承担 CRC 计算、主机读取并核对结果”组织软硬件协同流程。六级 CPU 负责数据准备、参数配置和运算调度，CRC 引擎按字节更新校验状态，结果邮箱提供统一的数据出口，使功能测试与性能测量在同一套系统中完成。

实际问题与设计目标

设计关注三个问题：处理器通过 MMIO 调用 CRC 引擎时的完整调用收益，输入长度对固定调用开销占比的影响，以及增加硬件 CRC 后的 SoC 资源变化。评估同时记录查表软件、硬件完整调用和引擎会话三种周期，以对应程序执行、软件调用和外设运行三个层次。

设计目标如下：

- 实现 RV32I 基础整数指令与 Zicsr 机器模式子集，支持异常、中断、mret、ecall、ebreak；
- 采用六级顺序单发射流水线、静态前端预判、三级前递、Load-use 停顿和 save 保持；
- 使用 16 KiB 指令 ROM 与 64 KiB 数据 RAM，保留 BRAM 输出寄存器和两拍读；
- 实现 CRC-32/ISO-HDLC MMIO 加速器、128 字结果邮箱和 JTAG/VIO 读取；
- 在真实 Z7-LITE_ES1 板卡上完成三次复位、六次扫描和 36/36 CRC 比对；
- 给出同器件、同时钟、同存储容量的软件/硬件性能与资源对照，并保留短输入不加速的真实结果。

2.1.2 系统架构与软硬件划分

系统采用 Harvard 存储组织，将取指通路与数据访问通路分开。六级流水线把操作数前递和执行运算分置于 EX0 与 EX1，通过前递仲裁、停顿及冲刷保持数据与控制的一致性。CRC 外设使用 32 位数据寄存器和 4 字节缓冲承接 CPU 写入，计算逻辑每拍处理一个字节。

软件实现选择 256 项查表 CRC 作为性能基线，并用逐位 CRC 交叉核对。主机使用 Python 标准库 zlib 生成独立参考值。硬件计算、固件自检、主机校验和板级指示共同构成结果核对流程，完整调用周期直接反映 CPU 与外设协同工作的实际成本。

• 完整调用计时：把配置、数据搬运、尾部处理、轮询和结果读取纳入主要加速比，直接衡量软硬件协同收益。

• 同配置资源评估：在相同器件、20 ns 时钟约束、16 KiB ROM、64 KiB RAM 和 VIO/邮箱尺寸下，分别实现软件 CRC 控制变体与硬件 CRC 版本，统计整体 SoC 资源净差。

• JTAG 板级测量：通过 128 字结果邮箱与小型 VIO 读取数据，三次 VIO 辅助复位产生六次稳定扫描，36 项 CRC 结果与独立 zlib 参考相符。

• 可复现的源码包：从空目录解压、GCC 重编译、七个 IP 重新生成、综合/实现/BIT，并给出板级采集与结果比对步骤。

系统工作于机器模式，提供整数运算、逻辑移位、条件分支、跳转、字节/半字/字访存以及 CSR 访问。应用程序放入 16 KiB 指令 ROM，常量、工作缓冲区和栈位于 64 KiB 数据 RAM。CRC32 外设、自由运行计数器和结果邮箱通过内存映射寄存器与 CPU 连接。 指令与 CSR 的规范定义参见文献[1-2]。

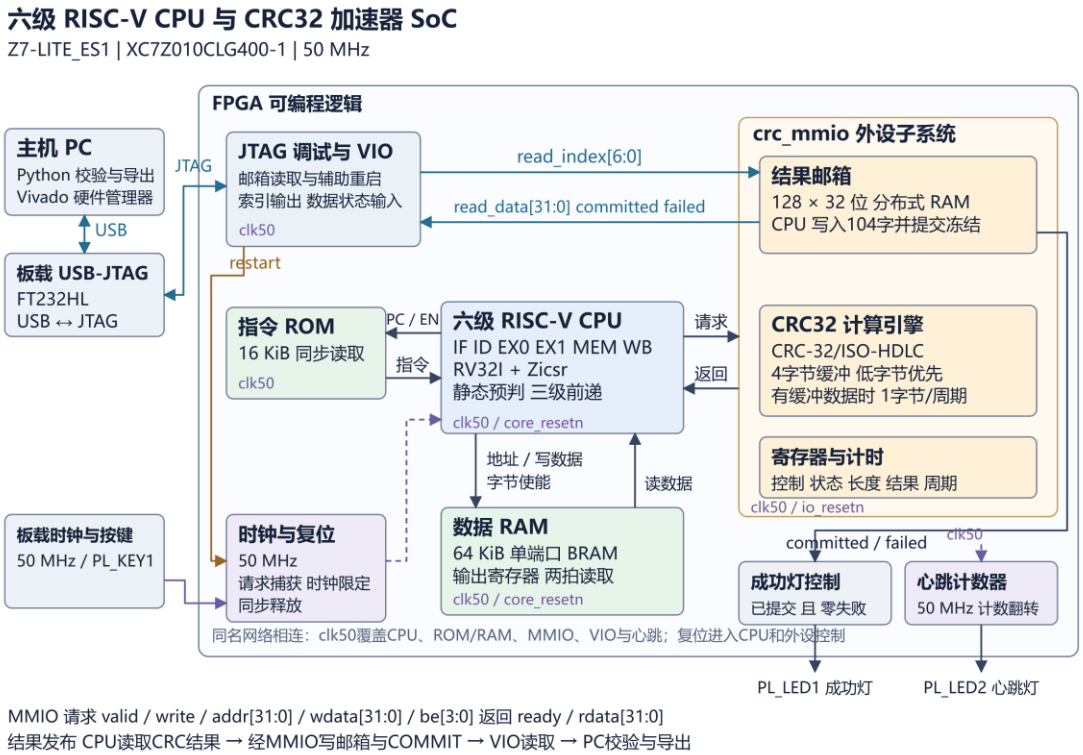


图 2-1 演示 SoC 系统框图

系统由主机、JTAG/VIO、六级 CPU、16 KiB 指令 ROM、64 KiB 数据 RAM、CRC32 MMIO 外设、128 字邮箱和板级指示组成。50 MHz 板级时钟经 Clocking Wizard 进入 CPU、CRC 外设和 VIO；Processor System Reset 产生受控复位。CPU 从 IROM 取指、从 DTCM 读写数据，MMIO 访问落到 0x8020_0000 段。裸机 C 固件生成测试数据，先做软件查表 CRC，再通过 MMIO 调用硬件 CRC，最后把 12 例结果写入邮箱并提交。主机通过 VIO 设置读索引、读取数据、检查提交/失败标志。

软硬件划分

功能	软件（RISC-V 裸机 C）	硬件（PL RTL）	主机（PC）
测试数据生成	xorshift32 / ASCII	—	独立 zlib 参考
软件基线 CRC	256 项查表 CRC	—	只读复核
硬件 CRC	配置、搬运、轮询、取结果	字节串行 CRC-32 引擎、长度/缓冲/状态/结果/计时	只读复核
结果发布	写 128 字邮箱、提交	邮箱存储与提交锁定、提交/失败标志	VIO 读 104 字
复位	重新初始化	异步捕获 + 两级时钟限定；VIO 辅助复位	发起/观察
指示	—	成功灯、心跳灯	用户目视

CRC 更新过程由固定的移位与异或运算组成，适合用组合逻辑展开每个字节的 8 次更新。CPU 负责按 32 位字读出数据并写入外设，硬件负责缓冲、逐字节计算和状态维护；主机独立生成参考值并解码邮箱。该划分把计算逻辑、调用开销和结果校验对应到清晰的模块边界。

2.2 硬件系统介绍

2.2.1 硬件整体介绍

项目	实现
板级时钟	N18，50 MHz，LVCMOS33，周期 20 ns；create_clock 约束 sys_clk_50
时钟 IP	Clocking Wizard 输入 50 MHz、输出 50 MHz；系统周期 20 ns
复位来源	板级 resetn（PL_KEY1/P16，低有效）+ locked + Processor System Reset；VIO restart 接辅助复位请求
复位契约	离钟复位请求立即捕获；两级时钟限定的复位进入 CPU/BRAM 控制逻辑；io_resetn 可无时钟断言；CPU 域复位期间 BRAM 内容保持
指令存储	I_ROM 16 KiB（4×RAMB36E1），同步单端口 ROM；pc[13:2]索引，指令有效状态与 PC 对齐
数据存储	DTCM 64 KiB（16 × RAMB36E1），单端口 RAM，输出寄存器；addr[15:2]索引；Load 两拍读；DONT_TOUCH 保留原语
地址范围	RAM 0x8010_0000–0x8010_FFFF；MMIO 0x8020_0000 起；越界不静默折回
结构审计	16 个 DTCM RAMB36E1 的 DOA_REG=1；4 个 ROM RAMB36E1 保留；组合环 0
配置属性	CFGBVS=VCCO，CONFIG_VOLTAGE=3.3（Z7-LITE_ES1 配套资料）

地址或镜像布局	访问	对应模块
0x80000000 起 16 KiB	取指	指令 ROM，保存启动代码与裸机程序
0x80100000–0x8010FFFF	读写	64 KiB 数据 RAM，存放常量、数据缓冲和栈
0x80200100–0x8020011C	寄存器	CRC 控制、状态、长度、数据、结果、计时及提交
0x80200200–0x802003FC	读写	128 个 32 位结果邮箱字
0x80200020、40、50	寄存器	既有显示、LED 及计数器接口

本板引脚与电源连接以用户配套硬件资料为准，索引见第 2.3.5.4 节；两项配置属性的含义见文献[3]。

2.2.2 机械设计介绍

系统采用现成 Z7-LITE_ES1 开发板，设计内容为 FPGA 内部的 CPU、CRC32 外设、存储与调试连接。本作品无独立机械结构、外壳或机械加工设计。

2.2.3 电路各模块介绍

2.2.3.1 流水线组织

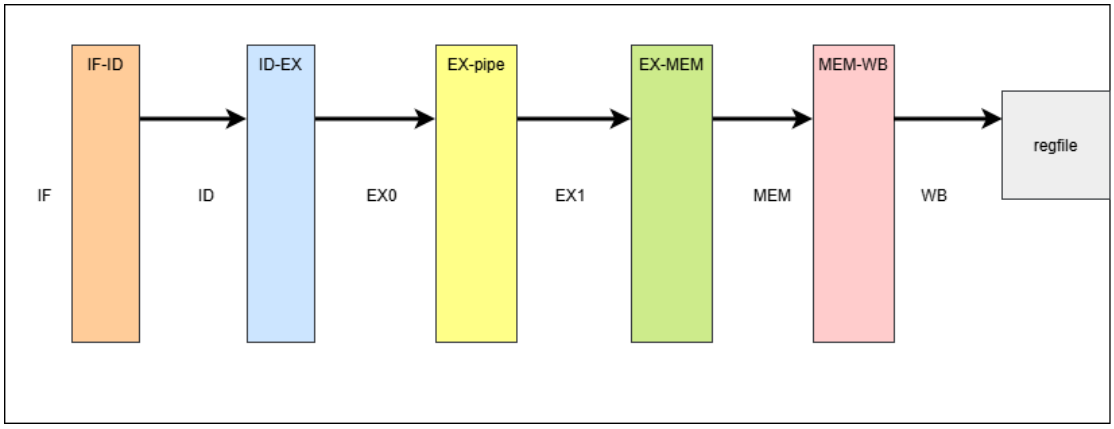


图 2-2 六级顺序单发射流水线与级间寄存器

CPU 采用 IF–ID–EX0–EX1–MEM–WB 六级顺序单发射结构，级间寄存器为 IF_ID、ID_EX、EX_pipe、EX_MEM、MEM_WB。将执行阶段拆成 EX0 和 EX1，是为了把前递/操作数筛选与 ALU/比较/分支校验分开，缩短关键路径。

阶段	主要工作	关键模块
IF	PC 更新、IBROM 取指、静态预判地址生成	npc、pc_judge、instr_minidecoder、IBROM
ID	指令字段解析、立即数生成、Regfile 读出、控制信号	Decoder、imm_gen、Regfile、Ctrl
EX0	前递仲裁、操作数筛选	EX_Forward、MEM_Forward、WB_Forward、Forward_Ctrl、EX_MUX
EX1	ALU/比较、分支校验、CSR、访存请求	ALU、Comp、Branch_Ctrl、csr_top、Store_Load

阶段	主要工作	关键模块
MEM	DTCM 返回、字节扩展、访存结果传递	MEM_DTCM、EX_MEM 及返回值通路
WB	写回数据选择、Regfile 写入	WB_MUX、MEM_WB

2.2.3.2 取指与静态分支预判

取指通路由 npc、pc_judge、IBROM 和 instr_minidecoder 组成。npc 保存顺序地址，pc_judge 在时钟边沿前完成地址选择，IBROM 同步读出指令并保存对应 PC，轻量预译码器据此生成下一拍的候选地址。图 2-3 展示这条基本通路。

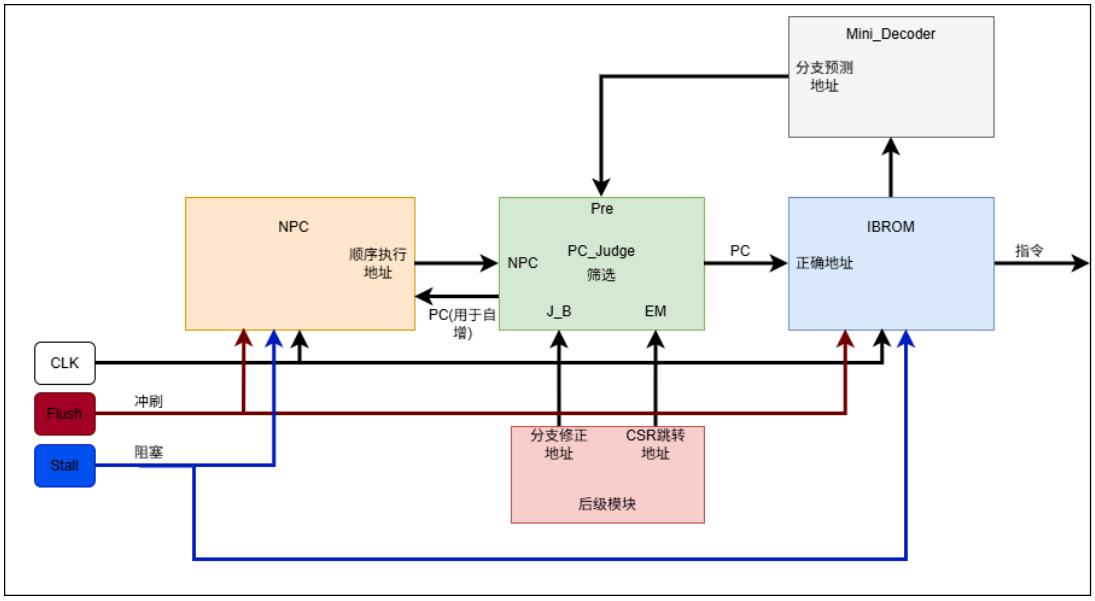


图 2-3 PC 仲裁与同步指令 ROM 的基本取指通路

（1）npc 与 pc_judge

npc 在正常顺序执行时按 PC 加 4 更新，并接收分支预判、分支修正和 CSR 控制转移对应的地址。阻塞时保持 PC；发生跳转后，以选定目标的 PC 加 4 作为后续顺序地址。复位入口为 0x80000000。

pc_judge 仲裁顺序 PC、预判 PC、分支修正 PC 和 CSR 目标 PC，并同时输出 PC 与 PC 加 4。PC 加 4 随指令进入后续流水级，供 JAL/JALR 链接地址回写及分支恢复顺序路径使用，从而复用前端的加法结果。当前选择顺序为分支修正、CSR 改向、待重放地址、静态预判、顺序地址；CSR 改向有效时，Branch_Ctrl 屏蔽普通分支修正，使系统控制转移获得实际优先权。

当后端改向与 ROM 停顿同时出现时，npc 保存尚未读取的目标地址，并置位 replay_redirect。停顿解除后，pc_judge 先选择该地址完成重放，再继续顺序取指，保证目标指令被正确读取。

（2）IBROM 与同步取指配合

IBROM 内部例化单端口 Block RAM IP 作为 16 KiB 指令 ROM，使用 PC[13:2] 寻址 4096 个 32 位字。时钟边沿采样地址并读出指令，同时保存该指令对应的 PC 和 PC 加 4。stall 有效时关闭 ROM 使能并保持 PC 信息。 BRAM 读时序及输出寄存器配置见文献[4-5]。

PC_Judge 模块起到了最关键的作用：其可以在上升沿到来前为 NPC 和 IBROM 两者准备好正确的地址。上升沿到来后，NPC 将 pc 值自增 4 递给 Judge，IBROM 根据 pc 值读出指令，以此循环。

正常连续取指时，上述地址选择与同步读出在相邻周期交叠进行，形成每周期提供一条指令的稳态取指节奏。复位后 fetched_valid 清零，IBROM 输出气泡；在有效读出后置位。若冲刷发生在停顿期间，先使保留的 ROM 输出失效，再由重放机制恢复目标路径，避免旧指令进入 ID。

（3）instr_minidecoder 与 IF_ID

接受 IBROM 输出的指令和 pc 值，仅针对 J 型和 B 型指令解码，进行计算后得到跳转的地址。分支预测规则为：JAL 一定跳转，直接输出对应 pc 值，JALR 因为静态预测无法进行预测，只能先顺序执行。对于 BRANCH 指令，如果是向后跳转（循环），则预测为直接跳转，如果向前跳转，则预测为不跳转顺序执行。输出 Pre 标志位和预测的 PC 值给 npc 和 pc_judge 模块，用于仲裁 pc 值。

预判方向由 B 型指令位移的符号位决定，J 型和 B 型目标均由当前 PC 与符号扩展后的立即数相加形成。EX1 随后校验实际分支条件，必要时纠正前端。

IF_ID 位于 IF 与 ID 之间，保存指令、PC 和 PC 加 4。复位或分支/CSR 冲刷时把指令清为气泡；停顿时保持；正常情况下将同步 ROM 输出及其 PC 信息一起送往译码级。

2.2.3.3 译码、寄存器堆与前递

译码与控制部分主要包括 Decoder、imm_gen、Ctrl 和 Regfile。运行过程如下：

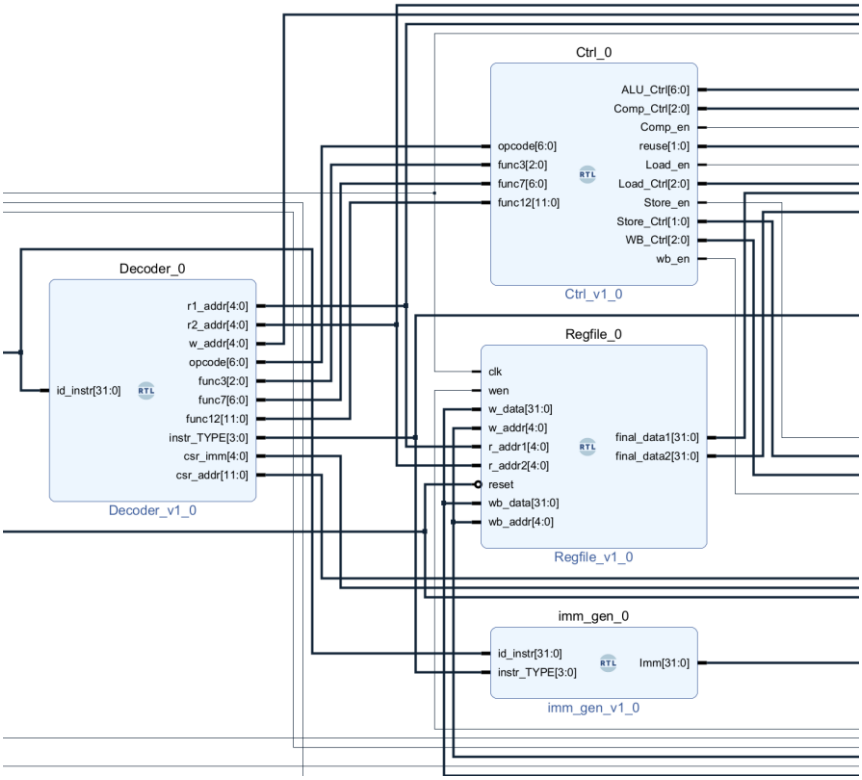


图 2-4 译码 控制与寄存器读出的基本数据通路

（1）Decoder 指令解析

指令进入 Decoder 后，会经过初期译码，将指令拆解成各个部分，输出给其它各个模块。输出信号如下：

字段	位宽	含义与去向
r1_addr / r2_addr	各 5 位	rs1、rs2 的有效寄存器地址，送 Regfile 和相关检测逻辑
w_addr	5 位	目的寄存器 rd 地址，随流水线传递到写回级
opcode / func3 / func7	7 / 3 / 7 位	运算与控制译码字段，送 Ctrl
func12	12 位	ECALL、EBREAK、MRET 的系统操作字段
instr_TYPE	4 位	指令类别，供立即数生成、操作数选择和前递数据选择
csr_imm / csr_addr	5 / 12 位	CSR 立即数或 rs1 编号字段，以及完整 CSR 地址

字段	位宽	含义与去向
instr_valid	1 位	非零指令标志，随流水线传递并限定有效系统操作

以上信号在经过筛选后才会输出，例如 I 型指令并没有 RS2 操作数，r2_addr 就会被置 0；B 型指令不需要写回，w_addr 就会被置 0。这样避免数据发生冲突和产生一些难以检测的错误。

CSR 立即数形式使用指令[19:15]作为零扩展立即数，r1_addr 置 0；寄存器形式则保留有效 rs1 地址。这样可区分“读取 x0”与“立即数字段为 0”等语义，前递检测仅针对实际读取的通用寄存器。

（2）imm_gen 立即数生成

用于生成立即数，接受 IF-ID 寄存器输出的指令以及 Decoder 输出的指令类型信号，提取对应指令的立即数段并进行拼接，生成对应的立即数并输出。

类型	立即数字段拼接	扩展方式
I 型	instr[31:20]	12 位符号扩展，用于立即数运算、Load 和 JALR
S 型	instr[31:25]、instr[11:7]	12 位符号扩展，用于 Store 地址计算
B 型	instr[31]、instr[7]、instr[30:25]、instr[11:8]、0	13 位符号扩展，最低位补 0
U 型	instr[31:12]、12 个 0	形成 32 位高位立即数，用于 LUI 和 AUIPC
J 型	instr[31]、instr[19:12]、instr[20]、instr[30:21]、0	21 位符号扩展，最低位补 0
R 型	0	操作数由寄存器提供

（3）Regfile 与 ID 读数旁路

主寄存器堆模块，定义了 RISCV 标准的 32 个寄存器，用于写回数据。接受 Decoder 输出的 RS1 地址与 RS2 地址，输出当前指令的两/一个源操作数。同时每个周期接受来自 WB 写回阶段的写回地址与写回数据，将其写入目标寄存器。

寄存器堆设两个组合读端口和一个上升沿写端口。x0 读出恒为 0，写入 x0 被抑制；复位时通用寄存器清零。写使能由 WB 的实际提交信号限定。

由于我们设计了六级流水线结构，前递统一在 EX0 阶段进行。会产生一种特殊的读后写数据冒险：如果 ID 阶段要读的数据在下个周期才被写回，但是前递要在下个阶段才能进行。这样就产生了数据冒险。

因此在 ID 读端口加入写回旁路：当本次有效 WB 的非零目的地址等于读取地址时，直接选用待写回数据；否则读取寄存器堆。该旁路与 EX0 前递配合，覆盖寄存器读数与写回相遇的时刻。

（4）Ctrl 与 ID_EX 控制传递

输出控制信号的模块。接受 opcode，func3，func7，func12 信号来进行判断应该输出哪些信号，以及对哪些信号置 0：

控制信号	位宽	作用
ALU_Ctrl	7 位	选择整数运算、比较结果复用及系统操作编码
Comp_Ctrl / Comp_en	3 / 1 位	选择比较关系并使能比较器
reuse	2 位	区分寄存器与立即数形式的 SLT 类比较复用
Load_en / Load_Ctrl	1 / 3 位	使能读取并区分 LB、LH、LW、LBU、LHU
Store_en / Store_Ctrl	1 / 2 位	使能写入并区分 SB、SH、SW
WB_Ctrl / wb_en	3 / 1 位	选择写回来源并限定目的寄存器更新

ID_EX 位于 ID 与 EX0 之间，寄存源操作数及其地址、立即数、PC、PC 加 4、目的寄存器地址、运算/访存/写回控制、CSR 字段和有效标志。冲刷清除当前指令的有效控制；停顿保持指令，同时可保存已经得到的前递操作数。

（5）三级前递与 Forward_Ctrl

为缩短执行阶段关键路径，将 EX 拆成 EX0 和 EX1：EX0 完成数据前递与操作数筛选，EX1 完成算术逻辑、比较、分支校验和访存请求生成。EX_Forward、MEM_Forward、WB_Forward 分别检测位于 EX1、MEM、WB 的生产者。

分别位于WB和MEM阶段，检测EX0阶段的指令的源操作数的地址(即rs1，rs2 地址)与本阶段的指令的写回地址是否相等。如果不相等，则不触发前递；如果相等，则触发前递，同时该模块根据指令类型发出前递数据识别的标志位。统一发送给 Forward_Ctrl 模块进行前递仲裁。

三个检测模块均要求生产者有效、写回使能有效且目的寄存器不为 x0。EX_Forward 还识别紧邻 Load 的结果尚未就绪情形，发出 Load-use 停顿与气泡控制。

Forward_Ctrl 接收 EX1、MEM、WB 的 ALU 结果、Load 结果、PC 加 4 和 CSR 读值，并按来源标志区分 DTCM 读值与已经完成的 MMIO 读值。

前递数据具有优先级。顺序为 EX1＞MEM＞WB。最终发出两个前递数据，以及两个前递使能，由 EX_MUX 做最后仲裁，筛选出传递给后续阶段的操作数。

当同一寄存器被多条在途指令写入时，选择距离消费者最近的有效生产者。ID_EX 在停顿期间还通过 retain_en1/retain_en2 保存已就绪的前递值；若最近生产者为尚未完成的 EX1 Load，则继续等待其结果。这样即使 MEM/WB 中的其他生产者先行离开，也能保留当前指令已经取得的操作数。

（6）EX_MUX 与 EX_pipe

该模块用于最后的操作数筛选。总共可归类为四种操作数：

1、寄存器 rs1 的数据，可以被前递数据修正

2、寄存器 rs2 的数据，可以被前递数据修正

3、立即数 IMM

4、PC 值

最终仅输出两个 ALU 操作数 A，B 与 COMP 操作数 A，B。

对于 ALU_A 操作数，仅当指令类型为 J 型，AUIPC，B 型时，选择 PC 值作为操作数，其余均选择 rs1 作为操作数；对于 ALU_B 操作数，仅当指令类型为 R 型时，选择 rs2 作为操作数，其余均选择立即数 IMM 作为操作数；

对于 COMP_A 操作数，仅当指令类型为 B 型、复用 SLT、SLTU、SLTI、SLTIU 时，选择 rs1 作为操作数，其余置 0。对于 COMP_B 操作数，仅当指令类

型为 B 型、复用 SLT、SLTU 时，选择 rs2 作为操作数，复用 SLTI、SLTIU 时，选择立即数 IMM 作为操作数，其余置 0。

EX_MUX 同时把经过前递修正的 rs2 作为 D2 送往后级，用于 Store 写入数据。EX_pipe 寄存 ALU/Comp 操作数、D2、PC 及全部后续控制，把操作数选择路径与执行运算路径分隔。Load 等待时保持当前执行指令；Load-use 气泡和分支/CSR 冲刷按相应控制清除。

2.2.3.4 执行、分支校验与 CSR

运算相关模块和分支校验以 ALU、Comp、Branch_Ctrl 为主，负责算术逻辑运算、大小比较、跳转目标生成、分支纠正。

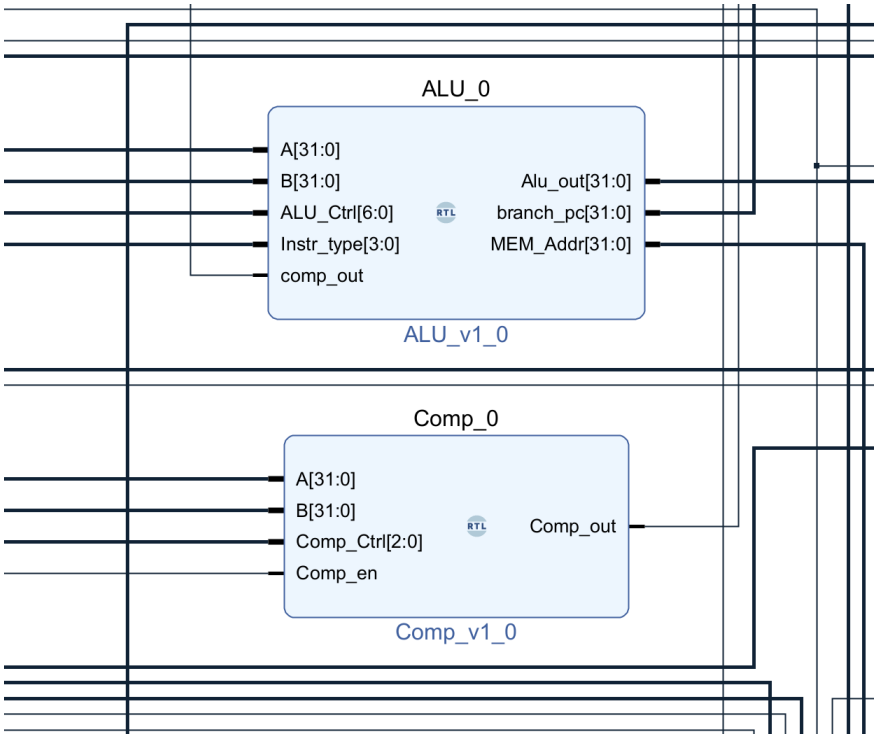


图 2-5 ALU 与独立比较器 Comp

（1）ALU 整数运算与地址生成

ALU 接收 EX_pipe 中的 32 位操作数 A、B 以及 ALU_Ctrl，执行加法、减法、按位与/或/异或、逻辑左移、逻辑右移和算术右移。移位量取 B[4:0]；SLT、SLTU 及其立即数形式复用 Comp 的 1 位判断，并在高 31 位补 0 后输出。LUI 通过立即数传递路径形成结果。

加法结果同时服务于数据地址和分支目标：Load/Store 使用 rs1 加立即数形成完整有效字节地址，条件分支使用 PC 加位移形成跳转候选。ALU 分别输出 Alu_out、branch_pc 与 MEM_Addr；组合逻辑为各输出设置默认值，使每条指令的辅助输出由当前控制确定。

（2）Comp 并行比较

比较单元，仅用于 branch 指令和部分运算指令复用，由于 branch 指令同时需要比较和运算，分别为：比较寄存器中的值，来判断是否跳转；进行加法运算，计算跳转地址。如果不单独为其设置一个 COMP 模块，因为需要四个操作数（两个寄存器的值，一个 pc 值，一个立即数），会使 ALU 的端口复杂化，打乱其两个运算数输入的结构。

因此 COMP 模块的加入可以使得 branch 的运算和比较同时运行，简化逻辑。

COMP 可执行的判断为：相等判断，不相等判断；有符号小于/无符号小于、有符号/无符号大于等于。

Comp_en 有效时输出所选条件；比较结果既供 Branch_Ctrl 判定跳转，也供 ALU 生成 SLT 类指令的 0 或 1 结果。

（3）Branch_Ctrl 分支校验与恢复

用于修正错误的分支预测以及输出正确的分支地址。内部的判断逻辑为：

JAL 指令：预测必然正确。不发出冲刷信号，跳转地址输出 32 位 0；

JALR 指令：前端按顺序地址继续，EX1 发出冲刷并输出 ALU 计算的目标，同时将目标最低位清零；链接地址仍使用随流水线传递的 PC 加 4。

BRANCH 指令：如果预测正确，则不发出冲刷信号，跳转地址输出 32 位 0；如果预测错误，分为两种情况：第一种为本来应该跳转，但是预测为顺序执行，则发出冲刷信号，跳转地址输出 ALU 计算得到的正确跳转地址；第二种为本来应该顺序执行，但是预测为跳转，则发出冲刷信号，跳转地址输出 pc＋4，即为顺序执行。

条件分支的预测方向由目标减去当前 PC 所得位移的符号确定，与前端按指令位移符号预测保持一致。CSR 控制转移有效时屏蔽普通分支 flush，由 CSR 给出恢复 PC；级间寄存器据此取消错误路径上的指令。

（4）csr_decode 与 CSR 原子读写

csr_top 包含 csr_decode 与 csr_regfile 两个子模块。它接收执行级操作码、寄存器操作数、5 位立即数、12 位 CSR 地址、PC、指令有效状态和停顿状态，输出 CSR 旧值、PC 改向信息及 cancel_current。

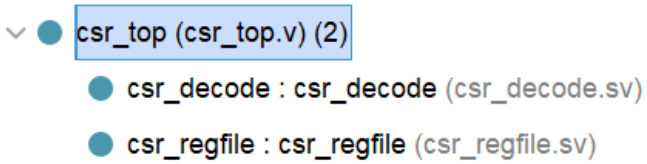


图 2-6 CSR 模块层次结构

csr_decode 采用纯组合逻辑，读取 csr_regfile 给出的旧值，根据指令形成新值与写使能。CSRRW/CSRRWI 直接写入操作数；CSRRS/CSRRSI 将旧值与操作数按位或；CSRRC/CSRRCI 将旧值与操作数的按位取反值相与。旧值送往后续写回路径，新值在该指令被接受的时钟边沿更新，形成同一条 CSR 指令内的读改写操作。

立即数形式将 5 位 uimm 零扩展到 32 位。CSRRS/CSRRC 在 rs1 编号为 0 时关闭 CSR 写使能，立即数置位/清位形式在 uimm 为 0 时同样仅执行读取。CSR 状态修改受 instr_valid 和 sys_stall 限定，流水线气泡与等待周期保持寄存器状态。

（5）csr_regfile 状态与异常控制

csr_regfile 采用时钟上升沿同步更新、组合读取的寄存器组织。模块通过完整 12 位地址选择寄存器，执行 CSR 写入、ECALL/EBREAK 异常进入和 MRET 返回，并输出恢复取指的 PC 及有效信号。mstatus、mtvec、mie、mip 与 mcause 等寄存器使用 SystemVerilog packed struct 描述字段。

地址	寄存器	当前实现行为
0x301	misa	标识 32 位整数处理器，读值 0x40000100

地址	寄存器	当前实现行为
0xF11–0xF15	mvendorid 、 marchid 、 mimpid 、 mhartid 、 mconfigptr	返回实现参数中的厂商、架构、实现、hart 及配置标识
0x300	mstatus	保存 MIE、MPIE 和 MPP 等机器模式控制字段
0x305	mtvec	保存异常入口基址及直接/向量模式
0x304 / 0x344	mie / mip	控制与保存机器外部、软件、定时中断的使能及待处理状态
0x340–0x343	mscratch、mepc、mcause、mtval	保存软件暂存值、恢复 PC、异常原因及相关状态
0xB00 / 0xB80	mcycle / mcycleh	读取 64 位周期计数器的低 32 位与高 32 位
0x320	mcountinhibit	CY 位控制周期计数暂停
0xB02 / 0xB82	minstret / minstreth	当前地址读值为 0

ECALL 和 EBREAK 分别记录异常原因为 11 与 3，mepc 保存当前指令地址；进入异常时保存 MIE 到 MPIE 并关闭 MIE。MRET 从 mepc 恢复取指，并按 MPIE 恢复 MIE。mtvec 采用直接模式时转向基址，向量模式下中断入口为基址加 4 倍原因编号。

中断控制按有效指令与安全边界接受请求：等待中的访存先完成；已经完成的访存和控制指令以其解析后的下一 PC 作为恢复位置，普通 ALU/CSR 指令可由 cancel_current 取消并保留当前 PC 供返回后重执行。待处理中断状态可在停顿期间锁存。演示固件通过 mstatus 管理临界区，并在 CRC 计时区间关闭中断。

2.2.3.5 访存与两拍读

访存相关模块由 Store_Load 与 MEM_DTCM 构成。其中 Store_Load 属于 EX1 阶段，MEM_DTCM 属于 MEM 阶段。两者协同完成数据 RAM 访问，并将访存类型、地址与返回值按流水级对齐。

（1）Store_Load 地址解码与访问分流

Store_Load 在 EX1 完成访存预处理，接收 ALU 产生的完整 32 位有效字节地址、源数据 D2 及 Load/Store 控制。先进行 RAM 与 MMIO 完整地址命中判断，再分别生成 BRAM 请求或 CPU 侧外设请求。

演示 RAM 范围为 0x80100000–0x8010FFFF。原有 MMIO 寄存器按 0x80200020、0x80200040、0x80200050 三个精确地址命中；CRC32、计数与邮箱接口位于 0x80200100–0x802003FF，再由外设寄存器逻辑细分地址。RAM 与 MMIO 请求互斥，外设访问不使能 DTCM。

有效地址沿 EX1 与 EX_MEM 保留到 MEM；MEM_DTCM 仅在 RAM 命中后计算 offset=有效地址−0x80100000，并取 offset[15:2]作为 16384×32 位 BRAM 的字索引。地址范围检查先于索引截取，边界外访问保持无写入，RAM 读回路径返回 0。

（2）Store 字节使能与写数据排列

模块根据访存地址最低两位 MEM_Addr[1:0] 判断存储位置，生成对应字节使能（wen）与对数据进行数据移位。

指令	addr[1:0]与 wen	写入数据及对齐规则
SB	00→0001；01→0010 10→0100；11→1000	D2[7:0]移至对应字节通道，其余字节不更新
SH	00→0011；10→1100	D2[15:0]写入低半字或高半字
SH	01 或 11→0000	奇地址不产生 RAM 写入
SW	00→1111	完整写入 D2[31:0]
SW	01、10 或 11→0000	非 4 字节对齐地址不产生 RAM 写入

IO_Wen 表示外设请求有效，IO_WB 标识 Load 结果来自总线；IO_write、IO_be 和 IO_wdata 描述访问方向、字节使能及写入数据。MMIO 仅在 valid 与 ready 同时有效的完成沿产生副作用，等待时保持请求内容；取消或复位会撤销尚未完成的请求。已完成的读值与 Load 的目的寄存器及控制一同寄存。

（3）MEM_DTCM 紧耦合数据存储

MEM_DTCM 采用紧耦合存储方式，CPU 直接连接片上数据 RAM。

内部例化单端口 Block RAM IP，演示配置容量 64 KiB、字宽 32 位、深度 16384，并启用 BRAM 输出寄存器。Store 在有效写入边沿按字节使能更新；Load 经过存储阵列与输出寄存器两个时钟边沿读出。数据 RAM 由 16 个 RAMB36E1 实现，DOA_REG 保持为 1。

MEM 阶段使用 EX_MEM 保存的完整访存地址、低两位偏移及 Load_Ctrl，对 BRAM 返回的 32 位字进行选择和扩展。复位会屏蔽访问控制，RAM 内容保留；固件启动代码负责清零 BSS 及准备运行数据。

（4）Load 字节选择与扩展

Load 与 Store 使用相同的字节地址排列规则。根据 mem_addr[1:0]选择字节或半字，并按 Load 类型完成符号扩展或零扩展：

指令	取值位置	32 位输出
LB	偏移 0/1/2/3 分别取[7:0]、[15:8]、[23:16]、[31:24]	所选 8 位数据符号扩展
LBU	偏移选择与 LB 一致	高 24 位补 0
LH	偏移 0 取[15:0]；偏移 2 取[31:16]	所选 16 位数据符号扩展；奇地址输出 0
LHU	半字位置选择与 LH 一致	高 16 位补 0；奇地址输出 0
LW	偏移 0 取[31:0]	完整 32 位字；其他偏移输出 0

（5）BRAM 两拍读取与 Load 等待

stall_Load 与 load_over 配合，将两拍 BRAM 读取纳入六级流水线。图 2-7 展示保持执行级、为后级插入气泡以及数据返回后的扩展路径。

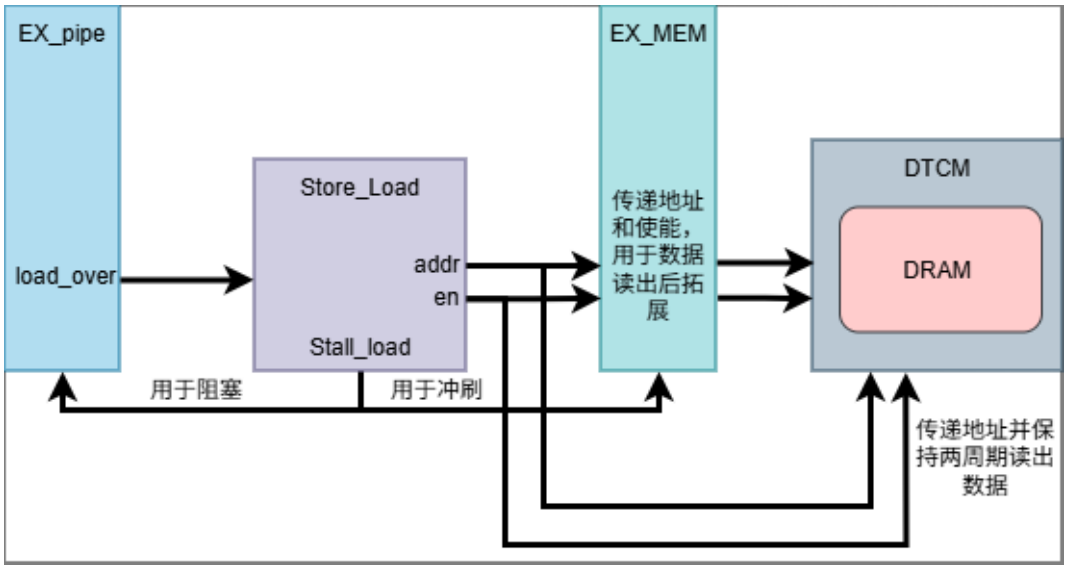


图 2-7 数据 BRAM 两拍读取与流水控制

第一个读取边沿之前，Store_Load 识别 RAM Load，且 load_over 为 0，因而拉高 stall_Load；地址与使能送到 BRAM，前端和 EX_pipe 保持，EX_MEM 插入气泡。

第一个读取边沿到来后，BRAM 完成阵列读取，EX_pipe 记录 load_over=1，当前 Load 的操作数与控制继续保留。该标志使下一周期的 stall_Load 解除，允许流水线在输出寄存器数据到达时前进。

第二个读取边沿，BRAM 输出寄存器给出数据，Load 的完整地址与控制进入 EX_MEM；MEM 阶段完成字节选择和符号/零扩展，随后经 MEM_WB 进入写回。两拍读取由流水线吸收其中一拍，RAM Load 本身增加一个保持周期。

当下一条 EX0 指令依赖该 Load 时，EX_Forward 进一步发出 Load-use 控制：保持 ID_EX、在 EX_pipe 插入气泡，等待 Load 结果可前递。相关指令随后取得 MEM 中的读值继续执行。两个操作数同时相关时，save_Load_use 与 ID_EX 的前递值保留逻辑共同维持已就绪数据。

2.2.3.6 回写、停顿与冲刷

（1）WB_MUX 与寄存器提交

回写部分主要由 WB_MUX 以及寄存器堆写入口构成。

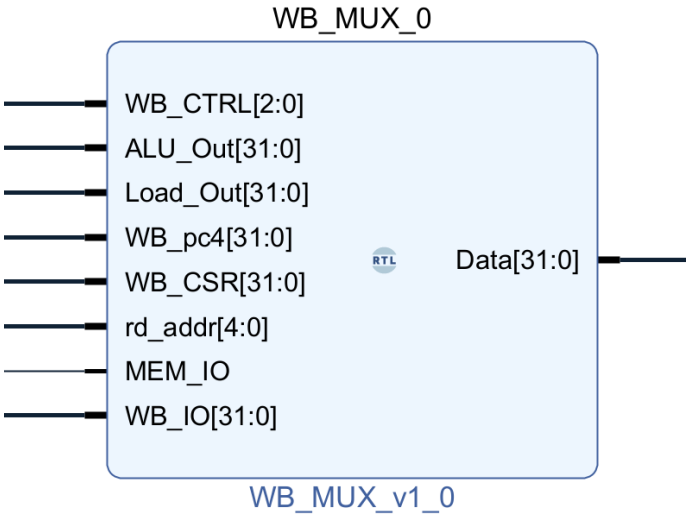


图 2-8 写回来源选择模块 WB_MUX

WB_MUX 接收 MEM_WB 寄存的 ALU_Out、Load_Out、WB_pc4、WB_CSR 和 WB_IO，分别对应整数运算、数据读取、链接地址、CSR 旧值和总线读值。WB_CTRL 选择写回来源；rd 为 x0 时输出 0。

当 WB_Ctrl 为 Load 标志位且 MEM_IO 置 0 时，选择 Load_Out；

当 WB_Ctrl 为 Load 标志位且 MEM_IO 置 1 时，选择 WB_IO；

下个周期上升沿到来时，数据写回到 Regfile，正式完成一条指令的执行。

MEM_WB 分别提供 Forward_valid 和 Wen_out：前者在保持期间保留待写回结果的前递资格，后者只有在未 stall、未 save 且指令有效时才允许寄存器提交。保持解除时执行一次写回并推进流水线。

（2）五个级间边界的控制

级间寄存器把数据与其所属指令的控制信号一起传递。冲刷通过清除有效或写入使能形成气泡；停顿保持正在等待的指令，同时让较早完成的后级工作按相应规则退出。各边界承担的任务如下。

级间寄存器	主要保存内容	保持与清除行为
IF_ID	指令、PC、PC 加 4	分支/CSR 冲刷清除指令；stall 或 stall_Load 保持
ID_EX	源操作数及地址、立即数、目的地址、运算/访存/CSR 控制、有效状态	冲刷清除；等待时保持指令，可由 retain 保存已经就绪的前递值
EX_pipe	ALU/Comp 操作数、D2、PC、访存与回写控制	分支/CSR 冲刷清除；两拍 Load 等待保持；Load-use 时插入气泡
EX_MEM	ALU/CSR/MMIO 数据、完整 Load 地址及类型、目的地址与写回来源	RAM 或 MMIO 等待时插入气泡；取消当前指令时清除；完成后采样结果
MEM_WB	全部写回候选数据、目的地址与写回使能	stall 或 save 保持；保持期间可前递，解除后一次提交

（3）Stall 与 save_Load_use

Stall 将 EX_Forward 的 Stall_Ctrl2 传递为 Stall2，驱动前端及 ID_EX 的相关停顿。Store_Load 的 stall_Load 另行覆盖 RAM 两拍等待和 MMIO ready 等待。两类等待分别对应“操作数尚未就绪”和“当前访存尚未完成”。

save_Load_use 检测两类停顿重合的情况，并在下一周期产生 save，使 MEM_WB 中的待写回数据额外保留一拍，供等待中的 EX0 指令使用。配合 ID_EX 保存已就绪前递值，可覆盖一个操作数等待 Load、另一个操作数已到 WB 的场景。save 随后自动解除，待写回结果继续提交。

分支纠正、CSR 改向、Load-use 和访存等待在各级按各自职责处理：错误路径被清除，尚未完成的访存保持，已完成的 MMIO 读值随目的地址进入后级。复位与取消通过有效控制限定寄存器写入和外设副作用。

（4）时钟与复位保护

演示系统由 clk50 提供 50 MHz CPU 与外设时钟。cpu_reset_sync 分别捕获外部复位、时钟失锁及控制器复位请求，cpu_reset_control 再通过时钟域内两级资格确认寄存器产生 CPU 复位。对外 io_resetn 在请求到来时即可断言，CPU 与 BRAM 控制状态在时钟边沿响应并在资格确认完成后释放。

该复位组织保留停钟期间的请求捕获能力，并将 BRAM 控制路径的复位转换到时钟域内处理。热复位通过清除取指有效状态和流水线控制重新进入 0x80000000，VIO 的 restart 经辅助复位输入触发同一运行重启流程。

2.2.3.7 指令与系统操作

类别	已实现指令或操作	用途
寄存器整数运算	ADD、SUB、SLL、SLT、SLTU、XOR、SRL、SRA、OR、AND	算术、比较、逻辑与移位
立即数整数运算	ADDI、SLTI、SLTIU、XORI、ORI、ANDI、SLLI、SRLI、SRAI	常量操作与地址计算
高位与 PC 相关	LUI、AUIPC	构造常量和 PC 相对地址
读写存储	LB、LH、LW、LBU、LHU；SB、SH、SW	字节、半字、字访问及扩展
分支与跳转	BEQ、BNE、BLT、BGE、BLTU、BGEU；JAL、JALR	控制转移与函数调用
系统操作	ECALL、EBREAK、MRET	异常入口与机器模式返回
CSR 访问	CSRRW、CSRRS、CSRRC、CSRRWI、CSRRSI、CSRRCI	状态和机器模式控制寄存器访问

2.2.3.8 数据通路信号汇总

表中汇总主要指令类别在操作数、执行与写回路径上的对应关系。各控制信号与该指令的 PC、目的寄存器和有效状态一起经过级间寄存器，按前述停顿与冲刷规则推进。

指令类别	ALU 操作数与结果	比较或访存控制	写回来源
寄存器运算	A=rs1，B=rs2	SLT/SLTU 使用 Comp 结果；其余按 ALU_Ctrl 执行	ALU_Out

指令类别	ALU 操作数与结果	比较或访存控制	写回来源
立即数运算	A=rs1，B=IMM	SLTI/SLTIU 以 rs1 与 IMM 比较；其余按 ALU_Ctrl 执行	ALU_Out
Load	rs1+IMM 生成完整地址	Load_en 及 Load_Ctrl；区分 DTCM/MMIO 并按宽度扩展	Load_Out 或 WB_IO
Store	rs1+IMM 生成完整地址	Store_en 及 Store_Ctrl；D2 携带前递后的 rs2，wen 选择字节	关闭寄存器写回
条件分支	A=PC，B=IMM 生成目标	Comp 比较 rs1 与 rs2；Branch_Ctrl 校验预判并恢复	关闭寄存器写回
JAL / JALR	JAL 目标由前端产生；JALR 为 rs1+IMM	JALR 目标最低位清零；必要时冲刷前端	PC 加 4
LUI / AUIPC	LUI 传递 U 型 IMM；AUIPC 为 PC+IMM	使用高位立即数及对应 ALU 控制	ALU_Out
CSR / 系统	CSR 寄存器形式用 rs1，立即数形式用 uimm	csr_decode 生成读改写；系统操作经 csr_regfile 改向	CSR 旧值或系统控制

2.2.3.9 处理器实现优化

CPU 以 EX0/EX1 分段组织前递与运算，减少单级中的操作数选择和执行逻辑深度。三级前递结合 Load-use 停顿与 save 保持处理数据相关；数据 RAM 输出寄存器配合两拍读控制稳定访存结果。这些结构与 50 MHz 约束共同形成当前实现，布局布线后的时序裕量见第 3.3 节测量表。

2.2.3.10 CRC 算法

CRC 采用 CRC-32/ISO-HDLC：反射多项式 0xEDB88320，初值 0xFFFFFFFF，最终异或 0xFFFFFFFF，输入字节低字节先送入，输出 32 位 CRC。主机用 Python zlib.crc32 独立计算 12 组向量，包括空输入、ASCII 123456789、长度 1/2/3/4/7/16/64/256/1024/4096 的确定性 xorshift32 数据。CRC 参数与 crc32 接口分别参见文献[6-7]。

CRC32 用于数据差错检测。标准字符串“123456789”的校验结果为 0xCBF43926，可作为软件、硬件和主机参考实现的共同核对点。

2.2.3.11 寄存器与握手契约

地址	访问	定义
0x80200100	W	CTRL：START/ABORT；ABORT 优先
0x80200104	R	STATUS：BUSY/DONE/ERROR
0x80200108	RW	字节长度；BUSY 写报错
0x8020010C	W	DATA：低字节先送入；缓冲满时 ready=0
0x80200110	R	最终 CRC32
0x80200114	R	ELAPSED：START 至最后字节，含喂数间隙
0x80200118	R	TICKS：自由运行计数器
0x8020011C	W	COMMIT：锁定邮箱
0x80200200–0x802003FC	RW	128 字邮箱；提交后只读

关键契约：所有 MMIO 寄存器使用字对齐 32 位访问；非对齐或非全宽访问完成握手后置 ERROR，不改变配置；所有副作用仅在 resetn & valid & ready 的上升沿发生；ready 只由寄存的缓冲占用状态决定，不依赖 valid、地址或 CPU 控制信号，避免组合环；START 接受非空长度置 BUSY，空长度立即 DONE 且 RESULT=0；重复 START 报错但保留当前任务；ABORT 优先；COMMIT 只在空闲且未提交时锁定邮箱。

2.2.3.12 CRC 引擎微结构

step_byte(c,b) 组合展开 8 次 (c>>1) ^ (c[0] ? 0xEDB88320 : 0)，每周期处理 1 字节。32 位 buffer_word 最多缓存 4 字节；buffered 记录剩余字节；remaining 记录总剩余；busy 期间每周期消耗 1 字节并递减。最后一个字节在同一周期产生 result = step_byte(crc, byte) ^ 0xFFFFFFFF，同时清 BUSY、置 DONE。elapsed 在 BUSY 期间递增，覆盖 START 接受到最后字节处理沿，包含 CPU 喂数间隙；ticks 为 32 位自由运行计数器，复位清零。

2.2.3.13 邮箱协议

索引	字段	含义
0	magic	0x43524331
1	version	1
2	clock_hz	50000000
3	ram_bytes	65536
4	case_count	12
5	error_count	固件累计错误数；正常为 0
6	total_words	104
7	executed_checks	7

每项测试占 8 个 32 位字，依次为用例编号、字节数、参考 CRC、软件 CRC、硬件 CRC、软件周期、硬件完整调用周期和引擎会话周期。邮箱前 8 字为配置与汇总，12 项测试共 96 字，因此一次读取 104 字即可取得完整测试记录。主机检查记录长度、编号、配置、CRC 和周期字段，并通过提交标志确认邮箱已经冻结。

2.2.3.14 JTAG 与 VIO

VIO 提供 read_index（输出 7 位）、restart（输出 1 位）、read_data（输入 32 位）、committed（输入 1 位）和 failed（输入 1 位）。主机先设置读取索引并提交 VIO 输出，再刷新输入取得对应邮箱字。restart 接入 Processor System Reset 辅助请求通路，实现时钟持续运行条件下的辅助硬件复位。VIO 探针配置见文献[8]。

2.2.3.15 错误处理

场景	行为
非法地址/方向/字节使能	完成握手、置 ERROR 并保持原配置
BUSY 写 LEN	置 ERROR；长度不变
重复 START	置 ERROR；保留当前任务

场景	行为
空长度 START	立即 DONE，RESULT=0
ABORT/START 同周期	ABORT 优先
握手完成前撤回请求	外设状态保持
COMMIT 后写邮箱	置 ERROR；内容不变
缓冲区未空	ready=0；其他访问最多等 4 周期

2.3 软件系统介绍

2.3.1 软件整体介绍

软件由 RISC-V 裸机 C 固件与 PC 主机脚本组成。固件在片上 ROM/RAM 中运行，生成确定性数据并执行逐位参考、查表软件 CRC 和 MMIO 硬件调用；主机负责环境准备、构建、仿真、JTAG/VIO 采集、独立 CRC 比对及周期表导出。主机、固件与硬件的关系见图 2-1。

2.3.2 软件各模块介绍

模块	输入	处理与输出
测试数据与参考	用例编号、字节数	生成 ASCII 或 xorshift32 数据；逐位 CRC 交叉核对
软件 CRC	缓冲区、长度	256 项查表；输出 CRC 与函数周期
硬件调用	缓冲区、长度	配置、搬运、轮询、读取；输出 CRC 与完整调用周期
结果邮箱	每例 CRC 与三种周期	写入 12 例、配置与错误数；提交后冻结
主机采集与验证	BIT/LTX、板卡身份、104 字邮箱	三轮复位与双扫描；独立 zlib 核对；导出 CSV

固件运行顺序为：启动并清零 BSS → 准备测试数据 → 计算逐位参考 → 采样查表软件 CRC 周期 → 调用 MMIO 硬件 CRC 并采样 → 比较三种

CRC → 写入邮箱 → 完成全部用例和边界检查 → 提交结果。主机在提交后读取邮箱，因此 JTAG 读取耗时位于函数计时之外。

2.3.3 从零开始复现

本章以 Windows 10 或 11 的 64 位环境为例，从软件安装和作品包解压开始，依次完成固件编译、IP 生成、仿真、综合实现、配套板卡运行及测量数据导出。示例工作目录为 C:\crc50_demo；可将工作目录整体替换为其他可写位置。命令在 PowerShell 中执行，每完成一步先核对该步输出，再继续下一步。

2.3.3.1 准备软件与配套硬件

安装 Python 3.12.x 并勾选 Add Python to PATH。安装 Vivado 2026.1，选择 Zynq-7000 器件支持，并安装随工具提供的 USB-JTAG 电缆驱动。准备 Z7-LITE_ES1 配套板卡、电源和 USB 数据线；该实测平台的器件为 XC7Z010CLG400-1，USB-JTAG 序列号为 210241793374。板载时钟为 50 MHz，配置属性采用 CFGBVS=VCCO、CONFIG_VOLTAGE=3.3。

打开新的 PowerShell 窗口，检查 Python 和 Vivado。将下例 Vivado 路径替换为实际安装目录。

```
python --version
$vivadoBin = 'D:\XILINX\2026.1\Vivado\bin'
& "$vivadoBin\vivado.bat" -version
```

预期 Python 显示 3.12.x，Vivado 显示 2026.1。RISC-V 工具链由后续 toolchain 命令下载并校验，固定为 xPack 15.2.0-1 及 GCC 15.2.0。固件采用 -march=rv32i_zicsr、-mabi=ilp32，并设置链接松弛和小数据寻址选项以匹配当前处理器。

2.3.3.2 校验与解压作品包

将作品总包 crc50_release_review2.zip 和同目录的.zip.sha256 文件放到本机。以下示例使用 C:\Downloads 作为接收目录，解压到新的 C:\crc50_demo\package 目录。

```
$zip = 'C:\Downloads\crc50_release_review2.zip'
$expected = ((Get-Content -LiteralPath "$zip.sha256" -Raw).Trim() -split '\s+')[0]
$actual = (Get-FileHash -LiteralPath $zip -Algorithm SHA256).Hash
```

```
if ($actual.ToLower() -ne $expected.ToLower()) { throw 'ZIP SHA256 mismatch' }
$work = 'C:\crc50_demo'
New-Item -ItemType Directory -Path $work | Out-Null
Expand-Archive -LiteralPath $zip -DestinationPath "$work\package"
Set-Location -LiteralPath "$work\package"
python -B -X utf8 verify_release.py --root . --extract
if ($LASTEXITCODE -ne 0) { throw 'Package verification failed' }
```

预期输出各组件的 COMPONENT_PASS 及 RELEASE_INTEGRITY_PASS。校验工具检查文件集合、SHA-256、ZIP 内部 CRC 和子包清单，并解压出 source、runtime、materials 和 github_gitee 四个目录。source 用于重建，runtime 用于配套实物运行，materials 保存原始测量与结果图，github_gitee 保存整理后的公开源码候选。

2.3.3.3 配置独立源码构建环境

```
$src = Join-Path $work 'package\source'
Set-Location -LiteralPath $src
$env:PYTHONDONTWRITEBYTECODE = '1'
$env:CRC_DEMO_STANDALONE = '1'
$env:CRC_DEMO_VIVADO_BIN = $vivadoBin
python -B -X utf8 run.py toolchain
if ($LASTEXITCODE -ne 0) { throw 'Toolchain setup failed' }
```

toolchain 命令将固定版本工具链保存到 source\tools\gcc，并核对官方工具链包 SHA-256。使用已解压的同版本工具链时，可先把 CRC_DEMO_GCC_BIN 设置为其 bin 目录，再执行 toolchain 命令。首次自动下载需要能访问工具链发行地址。

源码包中的 standalone.json 使构建直接使用 source 内的源码、约束、数据和脚本。构建工具为当前目录选择空闲短盘符，并将真实映射写入 evidence/drive_alias.json；后续 IP 与调试核生成沿用该映射。七个 IP 由 scripts 中的 Tcl 生成。

2.3.3.4 编译裸机固件并生成初始化镜像

```
python -B -X utf8 run.py firmware
if ($LASTEXITCODE -ne 0) { throw 'Firmware build failed' }
```

检查 firmware/out/firmware.elf、firmware.dis、firmware.map、rom.bin、ram.bin 以及对应 COE/HEX 文件。rom.bin 为 16384 字节，ram.bin 为 65536 字节。链接

脚本将.text 放到 0x80000000 起的指令 ROM，将只读常量、BSS 和栈放到 0x80100000 起的数据 RAM，栈顶为 0x80110000 并预留至少 4 KiB 栈空间。启动代码设置栈并清零 BSS，然后进入 main。构建同时检查指令、段布局和容量。

2.3.3.5 生成工程并完成仿真

依次执行工程生成、CRC 单元、数据存储、整机、故障模型和主机解析测试。下列循环遇到任何非零退出码都会停止。

```
foreach ($step in @('project','unit','memory','simulate','faults','controls')) {
    python -B -X utf8 run.py $step
    if ($LASTEXITCODE -ne 0) { throw "Step failed: $step" }
}
```

预期 CRC 单元 39 项检查通过；存储测试覆盖 64 KiB 首末地址、字节使能、访问宽度、边界与两拍读取；整机仿真完成三次运行、36 项 CRC 和 104 字邮箱；故障模型覆盖超时与 ERROR 状态；主机测试正确识别 9 类异常输入。每步记录位于 evidence 下对应的_result.json 及日志文件中。

2.3.3.6 完成综合实现并检查时序

```
python -B -X utf8 run.py implement
if ($LASTEXITCODE -ne 0) { throw 'Implementation failed' }
python -B -X utf8 run.py project
if ($LASTEXITCODE -ne 0) { throw 'Project refresh failed' }
python -B -X utf8 run.py report --source simulation
if ($LASTEXITCODE -ne 0) { throw 'Simulation report failed' }
```

实现产物为 evidence/demo.bit、demo.ltx 和 routed.dcp；Vivado 工程为 b/p/demo.xpr。通过 Vivado Open Project 打开该 XPR，打开 Implemented Design，在 Reports 中查看 Timing Summary、Utilization 和 DRC 报告。读取 WNS、TNS、WHS、THS、WPWS、TPWS；验收条件为建立、保持与脉宽裕量均大于等于 0，内部端点约束完整，路由错误为 0。约束中的时钟周期为 20 ns，数据 RAM 的 16 个原语输出寄存器应启用。 时序汇总字段解释见文献[9]。

报告给出的 6560 LUT、7892 FF、20 块 BRAM 和+2.824 ns 建立裕量来自本作品保存的实现。新一次布局布线应以该次生成的原始报告判定；资源表和时序表可分别在 evidence/utilization.rpt 与 timing_summary.rpt 中读取。

2.3.3.7 核对九项离线结果

以下命令按当前输入验证九项运行回执，预期九项均输出 PASS。仿真页面位于 report/simulation/index.html。

```
@'
from pathlib import Path
from scripts.evidence import validate_receipt
names = ('firmware','project','unit','memory','simulate',
        'faults','controls','implement','report_simulation')
for name in names:
    result = validate_receipt(Path.cwd(), name)
    print(name, result['status'])
    if result['status'] != 'PASS':
        raise SystemExit(1)
'@ | python -B -X utf8 -
```

2.3.3.8 连接配套板卡并读取 JTAG 身份

连接板卡电源，使用 USB 数据线接入板载 USB-JTAG 接口，确认电源指示正常。配套实物为 Z7-LITE_ES1、XC7Z010CLG400-1，USB 序列号 210241793374。使用 runtime 内与实测数据配套的 BIT/LTX 完成下面的批量运行流程。

```
$runtime = Join-Path $work 'package\runtime'
Set-Location -LiteralPath $runtime
python -B -X utf8 run_board.py verify
if ($LASTEXITCODE -ne 0) { throw 'Runtime file verification failed' }
python -B -X utf8 run_board.py preflight --vivado-bin $vivadoBin
if ($LASTEXITCODE -ne 0) { throw 'JTAG preflight failed' }
```

verify 输出 RUNTIME_HASH_PASS。preflight 在 captures 下创建新的时间目录，events.jsonl 记录目标与器件；对应目标 UID 为 Digilent/210241793374，器件 PART 为 xc7z010、IDCODE 为 0x13722093。核对实物型号、供电和连接后，执行以下命令，从刚生成的事件中读取目标及 DID 并建立本次身份文件。

```
$pre = Get-ChildItem -LiteralPath '.\captures' -Directory |
    Sort-Object Name | Select-Object -Last 1
$events = Get-Content -LiteralPath (Join-Path $pre.FullName 'events.jsonl') |
    ForEach-Object { $_ | ConvertFrom-Json }
$targets = @($events | Where-Object { $_.event -eq 'TARGET' })
$devices = @($events | Where-Object {
    $_.event -eq 'DEVICE' -and $_.fields.PART -eq 'xc7z010'
```

```
})
if ($targets.Count -ne 1 -or $devices.Count -ne 1) {
    throw 'Check the connected JTAG target and FPGA'
}
$identity = [ordered]@{
    pcb_revision='Z7-LITE_ES1'; USB_serial='210241793374'
    device_idcode=$devices[0].fields.IDCODE
    device_did=$devices[0].fields.DID
    target=$targets[0].fields.object
    physical_verified=$true; confirmed_for_this_capture=$true
    power_and_JTAG_verified=$true
    observed_at=(Get-Date).ToString('o')
}
$identityPath = Join-Path $runtime 'fresh_identity.json'
[IO.File]::WriteAllText($identityPath, ($identity | ConvertTo-Json),
    (New-Object Text.UTF8Encoding($false)))
```

该身份文件描述刚核对过的配套板卡。preflight 可同时显示 ARM 调试器件和 FPGA，FPGA 按 PART 筛选；将生成的身份文件与本次采集目录一并保存。

2.3.3.9 下载并完成三轮硬件测试

```
python -B -X utf8 run_board.py capture --identity .\fresh_identity.json --vivado-bin
$vivadoBin
if ($LASTEXITCODE -ne 0) { throw 'Board capture failed' }
```

该命令下载 runtime/demo.bit 并关联 demo.ltx，核对 VIO 与 5 个探针，然后执行三次 VIO 辅助硬件复位。每次等待 committed 置 1、failed 保持 0，再读取两份间隔至少 1 秒的 104 字邮箱。预期输出 CAPTURE_CRC_PASS。观察 PL_LED1 常亮、PL_LED2 闪烁，并把观察结果与采集日期一并记录。captures 中新目录保存六份 HEX、事件及工具日志。

配套运行 BIT 的 SHA-256 为 717bd8cfddeecf7753c8adcff68e0ec0072c0b9ca283c28613c2ac0dd3788d5b，LTX 为 facc5a5a95599c155598321dcd21d34b5d32f14b810fd7b6ef086fb8416ce6ba。第 2.3.3.6 节生成的文件属于本机重建产物，两组文件分别按各自路径和哈希保存。

2.3.3.10 复核 CRC 并导出周期表

在 runtime 目录执行以下命令，将刚采集的每轮首份扫描交给独立 zlib 参考校验，输出三个结果 JSON。双扫描一致性由 capture 命令同时检查。

```
$cap = Get-ChildItem -LiteralPath '.\captures' -Directory |
    Sort-Object Name | Select-Object -Last 1
foreach ($i in 0..2) {
    $dump = Join-Path $cap.FullName "run_${i}_scan_0.hex"
    $json = Join-Path $cap.FullName "run_${i}_validated.json"
    python -B -X utf8 skill/scripts/verify_mailbox.py --dump $dump --json $json
    if ($LASTEXITCODE -ne 0) { throw "Mailbox verification failed: $i" }
}
$env:CRC_CAPTURE_DIR = $cap.FullName
```

继续执行以下代码，生成 36 行 performance.csv。主要加速比始终使用硬件完整调用周期作分母；软件、完整调用和引擎会话三列均保留。

```
@'
from pathlib import Path
import csv, json, os
folder = Path(os.environ['CRC_CAPTURE_DIR'])
rows = []
for run in range(3):
    data = json.loads((folder / f'run_{run}_validated.json').read_text(encoding='utf-
8'))
    for source in data['dumps'][0]['rows']:
        row = dict(run=run+1, **source)
        row['software_us'] = row['software_cycles'] / 50
        row['hardware_full_us'] = row['hardware_total_cycles'] / 50
        row['engine_us'] = row['engine_session_cycles'] / 50
        rows.append(row)
with (folder/'performance.csv').open('w',newline='',encoding='utf-8-sig') as f:
    writer = csv.DictWriter(f, fieldnames=list(rows[0]))
    writer.writeheader()
    writer.writerows(rows)
print('CSV_COMPLETE',len(rows),'rows',folder/'performance.csv')
'@ | python -B -X utf8 -
```

核对标准字符串 CRC 为 CBF43926，测试总数为 12，每轮失败计数为 0。4096 字节参考记录的软件周期为 69653、硬件完整调用周期为 10295、会话周期

为 10250；0、1、2、3 字节的加速比低于 1 也属于完整结果。主机 JTAG 读取耗时位于函数计时之外。

2.3.3.11 查看本机重建工程的板级输出

完成第 2.3.3.6 节后，可在 Vivado 打开 source/b/p/demo.xpr，进入 Open Hardware Manager 并选择刚识别的 xc7z010 器件。通过 Program Device 选择 source/evidence/demo.bit 及对应 demo.ltx，完成下载后打开 VIO 面板，核对实例 vio 及 read_index、restart、read_data、committed、failed 五个探针。

将 restart 设为 0 并 Commit，再置 1 并 Commit 以触发辅助复位，保持至少 20 ms 后置 0 并 Commit。等待 committed=1、failed=0；read_index 使用十六进制、两位表示，从 00 到 67 共 104 项，每次 Commit 后 Refresh 读取 read_data。read_data 按十六进制显示：索引 00 为 43524331，01 为 00000001；02 为 02FAF080（50000000 Hz），03 为 00010000（65536 字节），04 为 0000000C（12 项），05 为 00000000（失败计数），06 为 00000068（104 字），07 为 00000007。索引 08 起按每例 8 字读取结果。

使用本机生成的 BIT 时，记录该文件与 LTX 的 SHA-256、工具日志及本次观察结果。配套批处理流程对应第 2.3.3.9 节给出的发布 BIT；本节用于在 Vivado 中直接检查本机重建产物。

2.3.3.12 常见运行问题处理

JTAG 列表为空时，核对供电、数据线、电缆驱动及 Hardware Server 连接；列出多个目标时按配套序列号选择正确连接。构建路径报调试核长度错误时，查看 evidence/drive_alias.json 并保持其短盘符映射；短盘符指向其他目录时先解决该目录占用。采集失败时根据 captures 内 stderr.log 与 events.jsonl 定位目标、探针或状态检查；修正后重新执行 preflight 并生成新的身份与采集记录。仿真步骤以自检终止标记和对应_result.json 为完成依据。

2.3.4 AI 辅助设计与可复用方法

2.3.4.1 AI 协作方式

团队在 RTL 与固件问题分析、测试脚本编写、数据交叉核对和报告整理中使用 AI 辅助。协作按“明确接口与输入输出、提出候选修改、执行验证、核对结果”的顺序进行。CRC 参考值由主机 zlib 计算，功能结果由自检仿真和板级邮箱给出，资源与时序由 Vivado 实际实现报告给出。

团队负责确定处理器结构、外设协议、验证要求和最终修改；AI 辅助完成重复检查、日志归纳及边界场景枚举。每次修改保留输入、命令和结果，便于追踪设计演进及复现测试。随作品提供的 AI 协作记录包含具体任务、提示词、工具输出和修正过程。

2.3.4.2 可复用方法

• 契约优先：先写清 valid/ready、完成/错误/中止、长度 0、尾部、撤回和复位的行为，再写 RTL 与测试。

• 独立参考：用 zlib 与标准字符串校验向量生成期望值，再对软件和硬件输出分别比较。

• 解析器异常输入测试：检查截断、多余记录、末项 CRC 错误、未知配置、错误计数、零周期和字节序。

• 三口径计时：软件周期、硬件完整调用、引擎会话；主要加速比只用完整调用。

• 资源对照：固定器件、时钟、CPU 和存储容量，记录两种实际实现及其全 SoC 资源净差。

• 源码重建：解压源码包，重新编译固件、生成 IP、执行仿真及布局布线，保存各步骤结果。

上述方法整理为随作品提供的 skill 目录，包含说明、邮箱校验脚本、接口模板和 CRC 测试示例。方法围绕同步 valid/ready 完成握手、MMIO 寄存器及固定布局邮箱组织，可用于类似的处理器外设集成与主机结果核对。

2.3.4.3 验证结果的形成过程

AI 协作形成的候选方案经过三层核对：仿真检查接口与数据行为，Vivado 实现检查约束下的物理结果，板级采集检查实际运行输出。独立参考、原始日志和结果哈希组成可追溯的验证记录。

2.3.5 术语与作品配套文件

2.3.5.1 术语

术语	含义
RV32I	RISC-V 32 位基础整数指令集
Zicsr	RISC-V 控制状态寄存器扩展
CRC-32/ISO-HDLC	反射多项式 0xEDB88320、初值/最终异或 0xFFFFFFFF 的 CRC32
MMIO	内存映射 I/O
VIO	Vivado 虚拟 I/O（JTAG 探针）
DTCM	紧耦合数据存储器（本设计 64 KiB）
IROM	指令 ROM（本设计 16 KiB）
WNS/WHS/WPWS	最差建立/保持/脉宽裕量
全 SoC 净差	相同约束下，两种完整 SoC 布局布线结果的资源差值

2.3.5.2 作品目录与复现对照

指南推荐结构	实际位置	说明
README.md	README.md	项目简介、复现入口、目录对照、许可证说明
src/	src/core/ 、 src/crc_mmio.sv 、 src/demo_top.sv	CPU/CRC RTL 与头文件
sim/	sim/	自检测试台

指南推荐结构	实际位置	说明
build/	scripts/、run.py	IP/综合/实现/报告脚本
board/	constraints/ 、 scripts/preflight.tcl 、scripts/hardware.tcl 、report/hardware/	约束、JTAG 预检/采集、板级结果
data/	data/、firmware/	初始化镜像、C 固件、启动/链接
skill/	skill/	可复用方法、脚本、模板、示例
report/	report/	设计报告、AI 记录、结果图

2.3.5.3 许可证与第三方声明

本队自研源码采用 MIT 许可证。AMD Vivado 生成的结构化连接、IP 与工具输出保留原 AMD 声明；厂商原理图、用户提供的赛事及设计资料不随公开源码再分发。完整版权署名、许可范围与第三方声明见源码包中的 license/ 与 docs/third_party.md。

2.3.5.4 配套资料与证据索引

本作品的接口说明、测试数据、AI 协作记录、Skill 与许可文件索引如下；这些文件是配套材料，不作为外部参考文献编号。

接口定义：docs/interface.md，包含 MMIO 寄存器、握手与 104 字邮箱协议。

硬件依据：docs/hardware_baseline_zh.md；原厂图纸为用户提供硬件包中的 Z7-LITE_Rev1_1.pdf，实物 ES1 与配套资料的对应关系按该说明保留。

板级测试：report/hardware/README.md，链接到实际采集回执、原始结果与三轮 CRC 比对记录。

性能与资源：总包中的 materials/records/evidence/agent_steps/step_06/report.md，配套 performance/ 与 resources/ 原始数据。

AI 协作记录：report/ai_collaboration.md。

可复用方法：skill/README.md 及其关联脚本、模板与示例。

许可与来源：license/license.txt、docs/third_party.md。

第三部分 完成情况及性能参数

3.1 整体介绍

实测平台为 Z7-LITE_ES1，器件 XC7Z010CLG400-1，板载 FT232HL 通过 USB-JTAG 连接主机。下载匹配的 BIT 与 LTX 后，采集脚本核对 VIO 及 5 个探针的方向、端口和宽度，执行三次 VIO 辅助硬件复位。每轮读取两份间隔至少 1 秒的 104 字邮箱，共得到六份扫描记录；36 项 CRC 与独立 zlib 参考一致，失败计数均为 0，双扫描内容一致。运行时 PL_LED1 点亮，PL_LED2 持续闪烁。

六份扫描记录保留原始邮箱字、采集事件、工具运行日志及 BIT/LTX 绑定信息，三次运行的周期字段一致。主机按相同用例序列比较仿真与板级结果，使功能验证、实现产物与性能表之间具有可核查的对应关系。

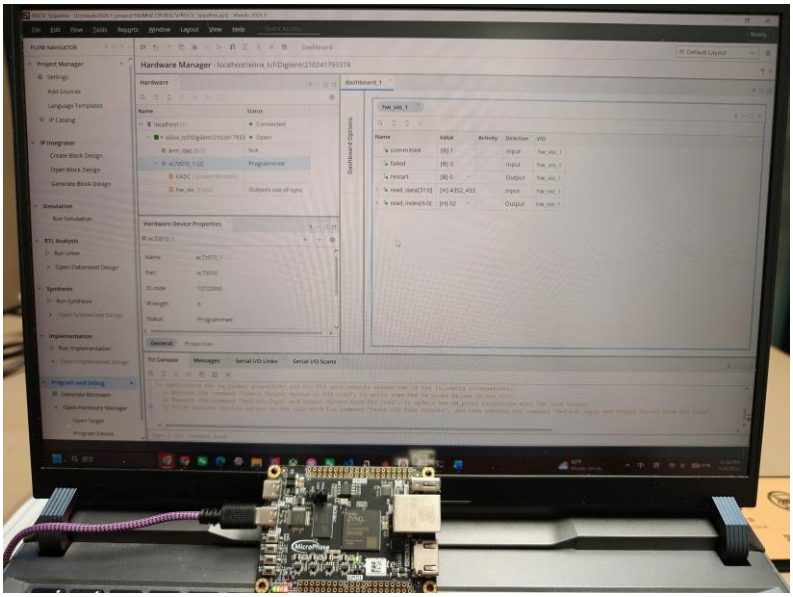


图 3-1 系统正面实物照片

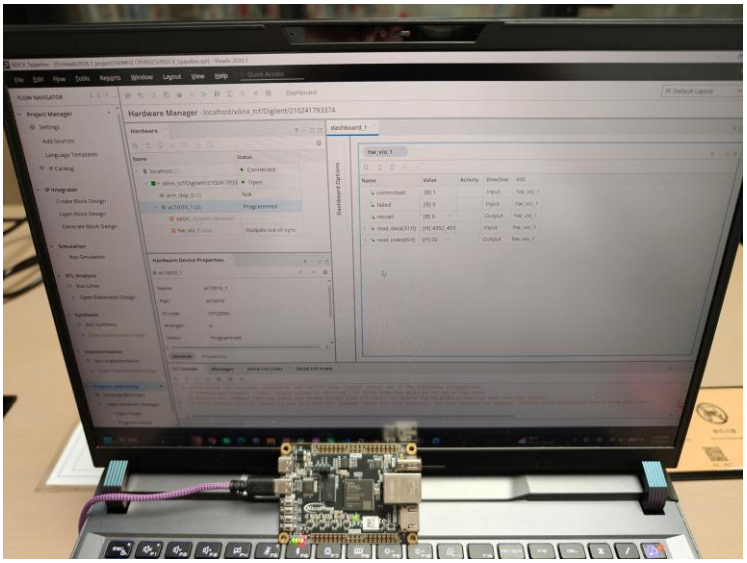


图 3-2 系统斜 45° 全景照片

3.2 工程成果

3.2.1 机械成果

本作品未设计独立机械部件，机械成果项不适用。板卡沿用 Z7-LITE_ES1 成品开发板。

3.2.2 电路成果

CPU、16 KiB ROM、64 KiB RAM、CRC32 MMIO、128 字邮箱与 VIO 调试接口已在 XC7Z010CLG400-1 上完成实现与板级运行。配套 BIT/LTX 由实际

构建生成，板级通过三次辅助复位、双扫描与独立 CRC 比对核对输出；运行时 PL_LED1 常亮、PL_LED2 闪烁。

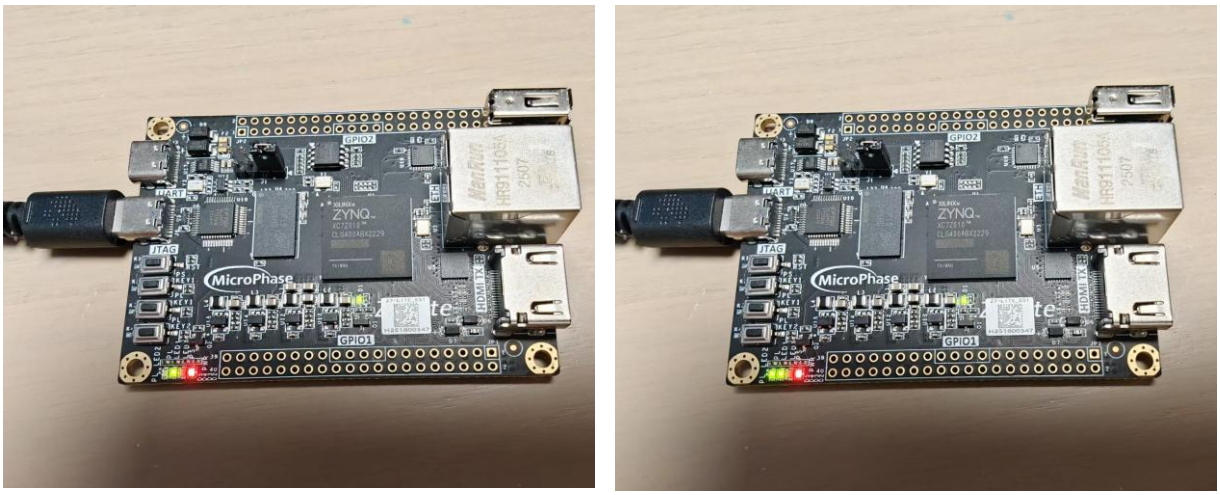
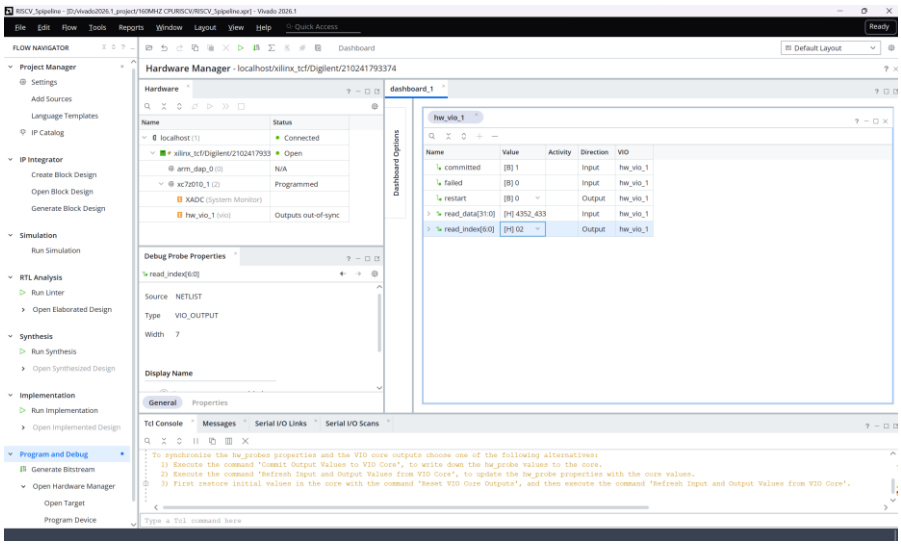
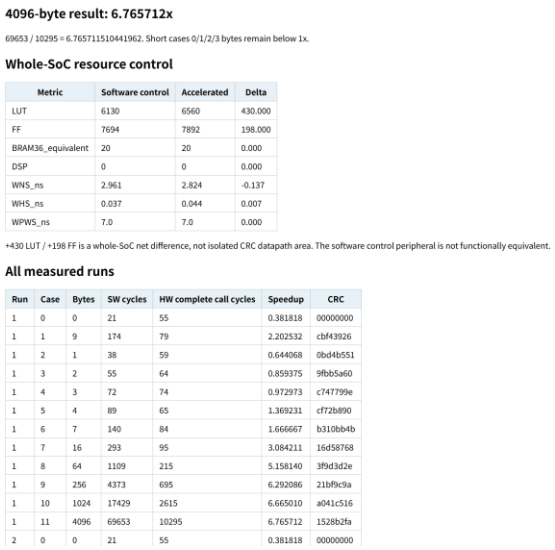


图 3-3 电路实物照片

3.2.3 软件成果

裸机固件、构建入口、板级采集脚本及主机验证脚本构成软件成果。固件编译生成 ELF、反汇编、链接映射和 ROM/RAM 初始化镜像；主机生成仿真页面、六份原始邮箱扫描、验证 JSON 与性能 CSV。具体生成命令及预期输出见第 2.3.3 节。





4096-byte result: 6.765712x

69653 / 10295 = 6.765711510441962. Short cases 0/1/2/3 bytes remain below 1x.

Whole-SoC resource control

Metric	Software control	Accelerated	Delta
LUT	6130	6560	430.000
FF	7694	7892	198.000
BRAM36_equivalent	20	20	0.000
DSP	0	0	0.000
WNS_ns	2.961	2.824	-0.137
WHS_ns	0.037	0.044	0.007
WPWS_ns	7.0	7.0	0.000

+430 LUT / +198 FF is a whole-SoC net difference, not isolated CRC datapath area. The software control peripheral is not functionally equivalent.

All measured runs

Run	Case	Bytes	SW cycles	HW complete call cycles	Speedup	CRC
1	0	0	21	55	0.381818	00000000
1	1	9	174	79	2.202532	cbf43926
1	2	1	38	59	0.644068	0bd4b551
1	3	2	55	64	0.859375	9fbb5a60
1	4	3	72	74	0.972973	c747799e
1	5	4	89	65	1.369231	cf72b890
1	6	7	140	84	1.666667	b310bb4b
1	7	16	293	95	3.084211	16d58768
1	8	64	1109	215	5.158140	3f9d3d2e
1	9	256	4373	695	6.292086	21bf9c9a
1	10	1024	17429	2615	6.665010	a041c516
1	11	4096	69653	10295	6.765712	1528b2fa
2	0	0	21	55	0.381818	00000000

图 3-4 主机采集与结果比对界面

3.3 特性成果

3.3.1 功能与故障验证

CRC 单元用 12 组独立 zlib 参考和 39 项检查，覆盖空输入、ASCII、1/2/3/4/7/16/64/256/1024/4096 字节、尾部、无 ABORT 连续任务、背压、撤回、重复 START、非法访问、复位/ABORT、计数回绕和提交后只读。数据修改/恢复测试中，4096 字节原 CRC 为 1528b2fa，修改第 17 字节 1 位后为 239d9555，恢复后再次为 1528b2fa。

存储测试覆盖首末字、末字节、首/末半字、四字节 lane、字/半字/字节访问、符号扩展、上下界外不别名、非对齐拒绝、复位保持和两拍读。整机测试做三次仿真复位，每次 12 组 CRC 和 104 字邮箱通过，共 36 例；每次监视 CPU 顺序清零 1025 个 BSS 字。故障注入覆盖 timeout（屏蔽 DONE）和 STATUS.ERROR 两个模型，CPU 实际写 errors、实际 ABORT、返回 0/正确 CRC，主机拒收故障邮箱。解析器负对照覆盖 9 类截断、额外记录、最后一项错误、错误配置和零周期等输入。

维度	结果
CRC 单元	12 组独立 zlib 参考、39 项检查通过

维度	结果
存储	64 KiB 地址/宽度/边界/两拍读通过
整机	三次仿真复位，36 例 CRC，104 字邮箱，BSS 清零
故障模型	timeout、STATUS.ERROR 两个模型符合预期
解析器负对照	9 类错误输入被拒收
实现	WNS +2.824 ns、WHS +0.044 ns、WPWS +7.000 ns
板级	3 次 VIO 复位、6 次扫描、36/36 CRC、失败 0
性能	12 组 × 3 轮；4096 字节 6.765712×
资源	LUT 6130→6560 、 FF 7694→7892 、 BRAM/DSP 不变

3.3.2 实现与时序

项目	值
器件	xc7z010clg400-1（Z7-LITE_ES1）
时钟	50 MHz / 20 ns（N18）
指令 ROM	16 KiB（4 × RAMB36E1）
数据 RAM	64 KiB（16 × RAMB36E1）
MMIO	0x8020_0000 段：CRC32 + 128 字邮箱 + VIO
实现结果	WNS +2.824 ns、WHS +0.044 ns、WPWS +7.000 ns；TNS/THS/TPWS 0
资源	LUT 6560、FF 7892、BRAM 20、DSP 0

项目	值
说明	50 MHz 内部静态时序的建立、保持与脉宽检查通过

XC7Z010CLG400-1 在 50 MHz 约束下完成布局布线，WNS 为+2.824 ns、WHS 为+0.044 ns、WPWS 为+7.000 ns，TNS、THS 和 TPWS 均为 0。片上 BRAM 共 20 块，16 个数据 RAM 原语的 DOA_REG 均为 1；内部时钟与端点约束检查通过，组合环为 0，12901 条可布线网络全部完成布线。ready 由 buffered 寄存状态生成，握手通路组合环检查通过。

3.3.3 软件基线与硬件完整调用

软件基线为同一固件中的 256 项查表 CRC。计数器包围软件函数和硬件函数，硬件完整调用包含 ABORT、长度及 START 配置、CPU 读内存、MMIO 搬运、尾部拼装、轮询、状态检查、结果读取以及调用和计数器采样开销。引擎会话周期从 START 接受到最后字节处理，包含 CPU 喂数间隙。主要加速比定义为软件周期÷硬件完整调用周期；时间按 50 MHz 换算，即周期数÷50 得到微秒。固件先生成数据并计算逐位参考，随后分别对软件调用和硬件调用计时，最后发布邮箱；JTAG 读取在提交后进行。

case	字节	软件周期	硬件完整周期	引擎会话	时间 µs（SW/HW/会话）	主要加速比
0	0	21	55	0	0.42 / 1.10 / 0.00	0.381818×
1	9（ASCII）	174	79	39	3.48 / 1.58 / 0.78	2.202532×
2	1	38	59	19	0.76 / 1.18 / 0.38	0.644068×
3	2	55	64	24	1.10 / 1.28 / 0.48	0.859375×
4	3	72	74	34	1.44 / 1.48 / 0.68	0.972973×
5	4	89	65	20	1.78 / 1.30 / 0.40	1.369231×
6	7	140	84	44	2.80 / 1.68 / 0.88	1.666667×
7	16	293	95	50	5.86 / 1.90 / 1.00	3.084211×

case	字节	软件周期	硬件完整周期	引擎会话	时间 µs（SW/HW/会话）	主要加速比
8	64	1109	215	170	22.18 / 4.30 / 3.40	5.158140×
9	256	4373	695	650	87.46 / 13.90 / 13.00	6.292086×
10	1024	17429	2615	2570	348.58 / 52.30 / 51.40	6.665010×
11	4096	69653	10295	10250	1393.06 / 205.90 / 205.00	6.765712×

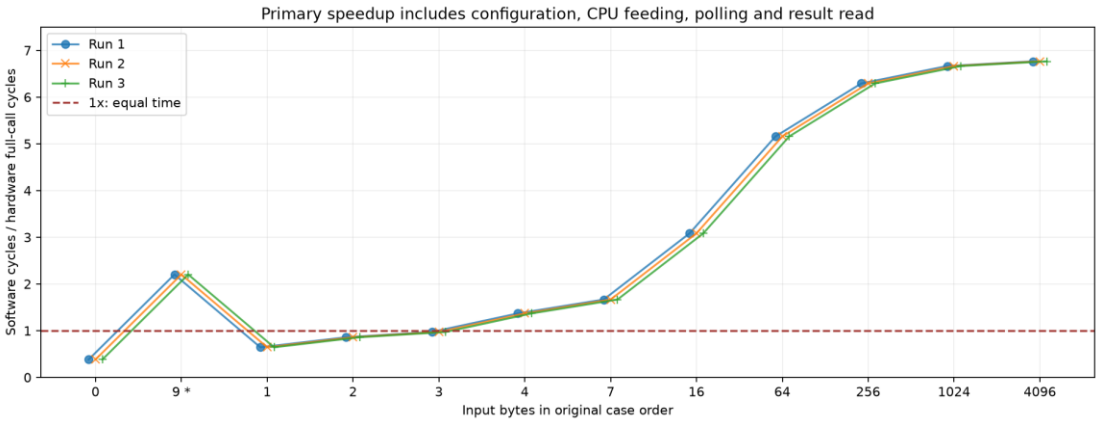


图 3-5 软件查表 vs 硬件完整调用（真实板级）

测量显示，0、1、2、3 字节用例的主要加速比分别为 0.381818、0.644068、0.859375 和 0.972973 倍，固定配置与结果读取开销在短输入中占比较高。4 字节用例达到 1.369231 倍，4096 字节用例达到 6.765712 倍。4096 字节的引擎会话为 10250 周期，完整调用为 10295 周期；会话包含 CRC 处理与 CPU 喂数节奏，反映当前软件供数方式下的协同运行时间。表中保留全部 12 组结果。

3.3.4 同配置资源对照

指标	资源基线（软件 CRC 控制变体）	加速版（硬件 CRC）	差值
LUT	6130	6560	+430
FF	7694	7892	+198
BRAM36 等效块	20	20	0
DSP	0	0	0
WNS / ns	+2.961	+2.824	−0.137

指标	资源基线（软件 CRC 控制变体）	加速版（硬件 CRC）	差值
WHS / ns	+0.037	+0.044	+0.007
WPWS / ns	+7.000	+7.000	0
TNS/THS/TPWS	0	0	0

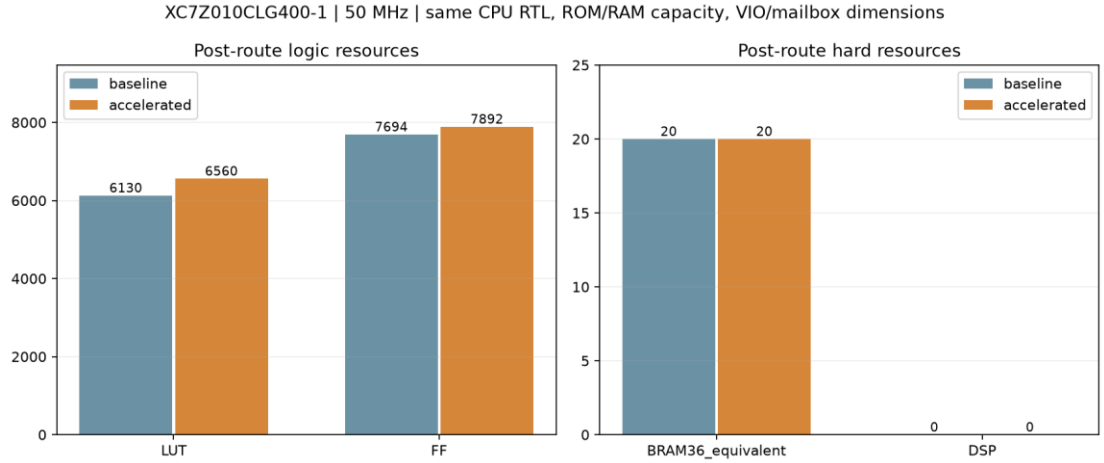


图 3-6 同器件资源对照（全 SoC 净差）

资源基线采用软件 CRC 控制变体：CRC 寄存器通路简化为固定响应，邮箱、计数器、CPU、存储容量和调试配置保持相同尺寸；其 hardware_crc 函数调用 software_crc。硬件 CRC 版本保留字节更新、长度、缓冲、状态及会话计时逻辑。两套构建分别生成固件并实际布局布线，资源表统计整个 SoC 的实现净差，包含外设控制、连接逻辑和 ROM 内容变化产生的综合结果。性能数据均取自硬件 CRC 版本中同一固件的两种调用。

结果分析

数据越长，单次配置、轮询和取结果的固定成本在总调用中的占比越低，因此主要加速比随测试长度上升。引擎在有缓冲数据时每拍处理 1 字节，CPU 按 32 位 MMIO 写入持续供数；4096 字节用例体现了该供数节奏下的整体收益。资源对照中，外设层 LUT 增加 510，CPU 层 LUT 减少 81，调试 Hub 层 LUT 增加 1，合计 SoC 净增 430 LUT；FF 净增 198，BRAM 容量保持 20 块。

3.3.5 验证数据与证据对应

验证数据按单元仿真、整机仿真、实现报告和板级采集分别保存。单元测试对应 CRC 寄存器及存储边界；整机测试对应固件执行和邮箱内容；实现报告对应资源、时序和 BRAM 结构；板级记录对应三次复位、六份扫描及实测周期。第 2.3.3 节给出各类数据的生成命令和检查位置。

本作品在 Z7-LITE_ES1 上实现六级 RISC-V CPU 与 CRC32 加速器的 50 MHz 协同运行。三次板级重跑的 36 项 CRC 全部匹配；4096 字节完整调用加速比为 6.765712 倍，SoC 净增 430 LUT 和 198 FF。裸机固件、验证脚本和源码重建步骤形成了可复现的软硬件协同流程。

第四部分 总结

4.1 可扩展之处

当前系统已验证片上 ROM/RAM、MMIO CRC32 与 JTAG/VIO 数据读取。后续可在保留现有接口契约和独立参考的基础上扩展测试数据长度分布、补充调用开销分解，并将邮箱核对方法复用于其他外设。若调整 CRC 并行度、供数方式或存储规模，需要重新检查握手、复位、固件容量、资源与时序，再做板级验证。

4.2 心得体会

本次设计将处理器、外设、固件和测量流程放在同一系统中核对。EX0 与 EX1 分工、三级前递及 BRAM 两拍读解决的是处理器执行路径的一致性；CRC32 的配置、搬运、轮询和取结果决定了应用侧的实际收益。短输入出现低于 1 的加速比，说明引擎局部速度不能替代完整调用的测量。采用独立 zlib 参考、故障模型和解析器负对照，可以检查正常结果及异常结果的处理方式。资源评估固定器件、时钟、存储和调试配置，并保留两个实际布局布线结果，使全 SoC 净差具有可比较的边界。 AI 辅助用于候选方案、脚本、日志归纳和边界场景枚举；我们负责结构、协议、验证要求和最终修改。输入、命令、原始输出与文件哈希共同支撑结论。把这些方法整理为 Skill 与可复现步骤，使后续检查能够沿相同流程重复执行。

第五部分 参考文献

[1] RISC-V International. The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture[EB/OL]. 20240411. (2024-04-11)[2026-11-04]. https://docs.riscv.org/reference/isa/v20240411/_attachments/riscv-unprivileged.pdf

[2] RISC-V International. The RISC-V Instruction Set Manual: Volume II: Privileged Architecture[EB/OL]. 20240411. (2024-04-11)[2026-11-04]. https://docs.riscv.org/reference/isa/v20240411/_attachments/riscv-privileged.pdf

[3] AMD. Vivado Design Suite Properties Reference Guide (UG912)[EB/OL]. 2026.1. (2026-07-20)[2026-11-04]. https://docs.amd.com/r/en-US/ug912-vivado-properties

[4] XILINX. 7 Series FPGAs Memory Resources User Guide (UG473)[EB/OL]. 1.14. (2019-07-03)[2026-11-04]. https://docs.amd.com/v/u/en-US/ug473_7Series_Memory_Resources

[5] XILINX. Block Memory Generator v8.4 LogiCORE IP Product Guide (PG058)[EB/OL]. 8.4. (2021-08-06)[2026-11-04]. https://docs.amd.com/v/u/en-US/pg058-blk-mem-gen

[6] COOK G. Catalogue of parametrised CRC algorithms with 17 or more bits: CRC-32/ISO-HDLC[EB/OL]. (2024-12-11)[2026-11-04]. https://reveng.sourceforge.io/crc-catalogue/17plus.htm#crc.cat.crc-32-iso-hdlc

[7] Python Software Foundation. zlib — Compression compatible with gzip[EB/OL]. Python 3.12 documentation. [2026-11-04]. https://docs.python.org/3.12/library/zlib.html#zlib.crc32

[8] XILINX. Virtual Input/Output v3.0 LogiCORE IP Product Guide (PG159)[EB/OL]. 3.0. (2018-04-04)[2026-11-04]. https://docs.amd.com/v/u/en-US/pg159-vio

[9] AMD. Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906)[EB/OL]. 2026.1. (2026-06-23)[2026-11-04]. https://docs.amd.com/r/en-US/ug906-vivado-design-analysis